vol-surface-engine — Cours complet du projet

Lecture guidée : architecture, code, mathématiques, et ce qu'il faut changer pour la crypto.

Version du 28 octobre 2026 — état vérifié du dépôt et de la base de production (Mac mini).

Table des matières

- 0. Comment lire ce document
- 1. N0 — Les idées générales
- 2. N1 — Cartographie du moteur C++
- 3. N2 — Le moteur C++ en détail, fonction par fonction
- 4. Le pipeline de données (Python + SQL)
- 5. Modélisation : taux, dividendes, forward — et comment passer à la crypto
- 6. Cours SSVI, exercices et questions d'entretien
- 7. Feuille de route et critères d'acceptation
- 8. Annexes

0. Comment lire ce document

Ce document est construit **par granularité croissante**, pas par ordre d'apparition du code. Il se lit dans l'ordre, mais chaque niveau est autonome : si tu ne veux que la vue d'ensemble, tu lis la partie 1 et tu t'arrêtes.

Niveau	Nom	Ce que tu obtiens	Partie
N0	Idées générales	Les schémas, les deux moitiés du système, le trajet d'une donnée d'un bout à l'autre	1
N1	Cartographie	Chaque structure de données, chaque fonction par son nom, qui appelle qui. On ne rentre pas dans les corps	2 (C++) et 4 (données)
N2	Détail	Le corps de chaque fonction : maths, entrées/sorties, pièges, complexité	3 (C++) et 4 (données)
N3	Cours et entretien	La théorie SSVI démontrée, les exercices, les questions d'entretien avec réponses	6
N4	Décisions	Rates, dividendes, forward : ce qu'on a choisi, pourquoi, comment basculer en crypto	5

Ordre de lecture recommandé : 1 → 2 → 3 → 5 → 4 → 6. La partie 6 (cours) se relit seule avant un entretien.

Conventions typographiques :

- `code` : un identifiant du dépôt (fichier, fonction, champ).
- Les blocs ASCII sont des schémas ; ils sont volontairement alignés en police fixe.
- « **mesuré** » = lu sur la machine ou sorti d'un test. « **constaté dans le code** » = lu dans les sources. Rien n'est affirmé de mémoire.

---

1. N0 — Les idées générales

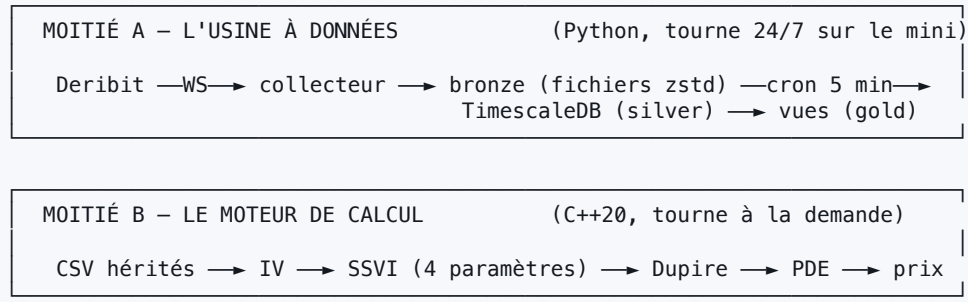
1.1 Le problème en une phrase

> On veut, à chaque instant, **une fonction** qui donne le prix (et la volatilité implicite) de n'importe quelle option sur BTC ou ETH, pour n'importe quel strike et n'importe quelle maturité — et qui ne viole aucune contrainte d'arbitrage.

Le marché ne cote qu'un nombre fini d'options : ~1 700 instruments BTC + ~1 700 ETH, sur 27 échéances. Entre deux strikes, il n'y a rien. Le modèle sert à **combler les trous de façon cohérente** (et à pricer des produits qui n'existent pas encore).

1.2 Les deux moitiés du système

Le dépôt contient deux systèmes qui ne se parlent pas encore. C'est **le** fait structurel à comprendre avant tout le reste.



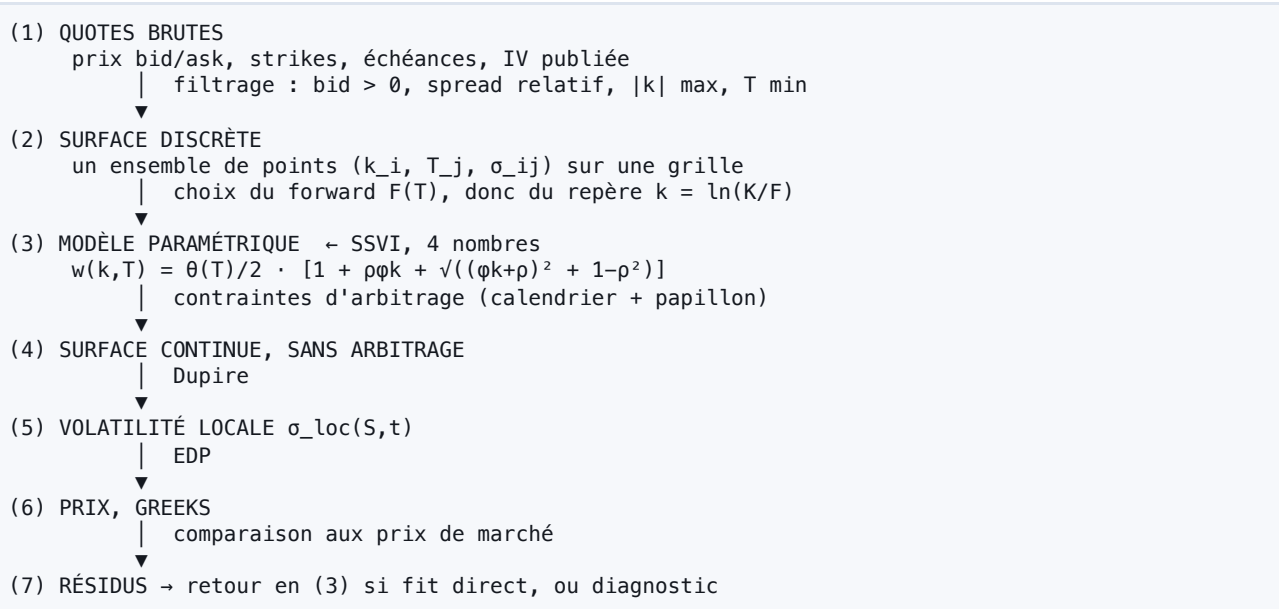
Pourquoi c'est important : **tout diagnostic est ambigu** tant que la coupure existe. Un mauvais fit peut venir du modèle, du solveur, du forward, des unités, ou du fait que le moteur lit un fichier vieux de six semaines. Fermer cette coupure est le chantier n°1.

1.3 Le trajet d'une donnée, de bout en bout



1.4 La chaîne de calibration, vue comme un pipeline de transformations

C'est le schéma à avoir en tête pour répondre à « comment marche ton projet ? » en entretien.



Chaque flèche est un endroit où l'on peut se tromper, et chaque erreur produit des symptômes différents. Un tableau de diagnostic est donné en §6.9.

1.5 Où tourne quoi

Composant	Machine	Démarrage	Langage	État
collecteur Deribit	Mac mini, Docker	`restart: unless-stopped`	Python	**healthy, 2 semaines**
TimescaleDB	Mac mini, Docker	`restart: unless-stopped`	SQL	**healthy**
`load_marks` (bronze→base)	Mac mini, cron 5 min	`ops/mini-load-marks.sh`	Python	actif
garde-fou stack / alertes / backups	Mac mini, cron	`ops/*.sh`	shell+Python	actifs
moteur de calibration	MacBook (natif)	manuel	C++20	compile, non raccordé
travail d'analyse	MacBook	tunnel SSH + parquet	Python/Jupyter	actif

1.6 Les trois invariants du système, énoncés simplement

1. **Rien n'est perdu.** Le message brut est écrit avant tout parsing. Si on se trompe dans la façon de lire, on rejoue — on ne recollecte pas. (Bronze = 13 Go pour 15 jours.)
2. **Rien n'est dupliqué.** Une ligne identifiée par `(instrument, horodatage du fournisseur)`. Rejouer une plage ne crée jamais de doublon. C'est ce qui rend un rattrapage inoffensif.
3. **Rien n'est silencieux.** Chaque rejet a une raison, chaque job écrit son statut, chaque collecteur expose sa fraîcheur. Les deux bugs les plus coûteux du projet (unités d'IV, référence de spot) étaient précisément des **silences** : un chiffre faux et crédible.

1.7 Vocabulaire minimal

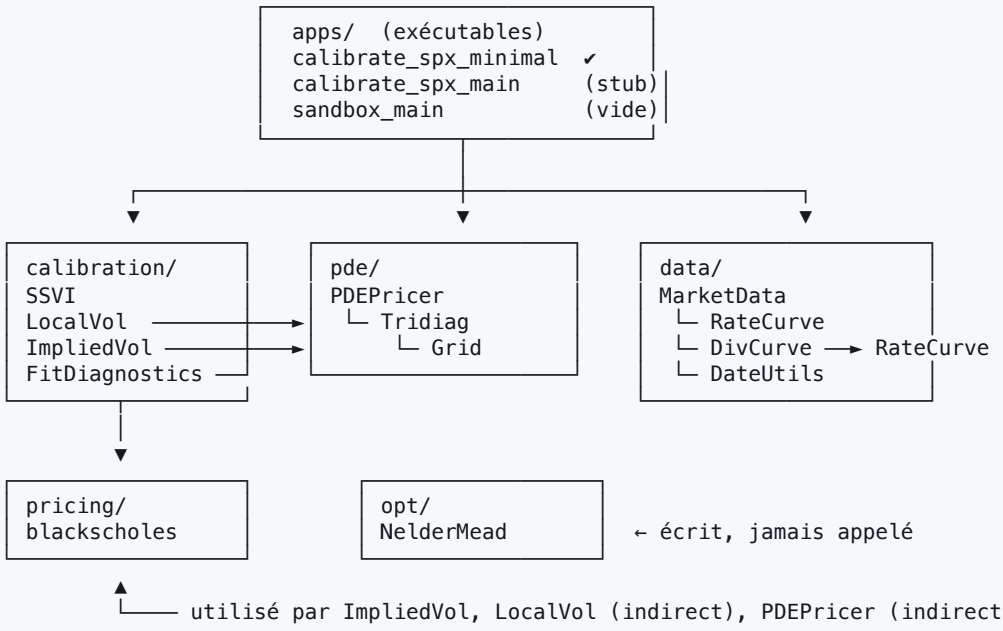
Terme	Définition opérationnelle dans ce projet
IV (vol implicite)	la volatilité σ qui, injectée dans Black-76, redonne le prix de marché. Unité : **pourcentage** (35,6 = 35,6 %)
forward F(T)	le prix d'achat à terme de l'actif pour l'échéance T. Repère de tous les calculs de smile
k	log-moneyness, $k = \ln(K/F)$. k=0 ⇒ option à la monnaie (ATM)
w	variance totale, $w = \sigma^2 \cdot T$. C'est la quantité qui s'additionne dans le temps
SSVI	*Surface SVI* : une famille paramétrique de surfaces $w(k,T)$ avec 3-4 paramètres, construite pour être sans arbitrage
η/η_max	le « thermomètre » : 1 signifie « le modèle voudrait plus de liberté que l'arbitrage n'en permet »
Dupire	la formule qui transforme une surface d'IV en volatilité locale σ(S,t)
PDE / EDP	l'équation aux dérivées partielles de Black-Scholes généralisée, résolue numériquement sur une grille
bronze / silver / gold	brut immuable / normalisé en base / vues de lecture
option inverse	option réglée en BTC ou ETH (et non en dollars). Le payoff est divisé par S_T

2. N1 — Cartographie du moteur C++

Objectif de cette partie : pouvoir dire, sans lire un seul corps de fonction, **ce qui existe, à quoi ça sert, et qui appelle qui**. C'est le niveau qu'on attend de toi quand on te demande « montre-moi l'architecture de ton moteur ».

2.1 Graphe de dépendances entre modules

La flèche signifie « dépend de » (un `#include`). Tout ce qui est en haut ne connaît rien de ce qui est en bas.



Deux choses à remarquer :

1. `calibration/` est le **centre de gravité** : SSVI, Dupire et le diagnostic y vivent. `pde/` ne connaît pas SSVI — il reçoit une `std::function` de vol locale. C'est un bon découpage : on peut pricer avec n'importe quelle vol locale, SSVI ou non.
2. `opt/NelderMead` est une **feuille morte** : déclarée dans le header avec la signature `minimize(f, Simplex&, NMParams)`, définie dans le `.cpp` avec `minimize(f, const vector&, NMParams)`. Les deux signatures ne correspondent pas ; rien ne l'appelle, donc le linker ne proteste pas. À supprimer ou à réparer, pas à « réutiliser » en l'état.

2.2 Catalogue des structures de données

Chaque structure est un **contrat**. Savoir qui la remplit et qui la lit, c'est savoir comment les modules se parlent.

Structure	Champs	Remplie par	Lue par
`pricing::BSParams`	`S, K, T, r, q, sigma`	appelant (main, impliedVol)	`d1, d2, callPrice, putPrice`
`calibration::IVResult`	`sigma, iterations`	`impliedVol` (succès)	main, runner
`calibration::SSVIParams`	`rho, eta, gamma, nu`	`calibrateSSVI`	tout le reste (w, ϕ, localVol, diagnostics)
`SliceFitSummary`	`T, n_points, rmse, max_abs_error`	`diagnoseSSVIFit`	runner
`FitDiagnostics`	`rmse_total, max_abs_error, eta_ratio, slices[]`	`diagnoseSSVIFit`	runner
`data::OptionQuote`	`K, T, isCall, exercise, bid, ask, volume, openInterest` (+ `mid()`)	`loadQuotes`	main, runner
`data::MarketData`	`spot, rates, divs, calls, puts`	`MarketData::load`	main, runner
`data::RateCurve`	`tenors[], rates[]`	`fromCSV`, constructeur	`rate, discount, forward`
`data::Dividend`	`T, alpha, beta`	`fromCSV`	`pvDividends`, `inInterval`
`data::DivCurve`	`divs[]`	`fromCSV`	`pvDividends`, `forwardDiscrete`
`pde::Grid`	`x[], S[], dx, N`	`make_uniform_grid`	`assemble_cn`, `apply_boundary`, interpolation finale
`pde::Tridiag`	`lower[], diag[], upper[]`	`assemble_cn`	`thomas_solve`
`pde::PDEParams`	`N, M, width, theta_cn`	défaut / appelant	`price`
`pde::PDEResult`	`price, delta, gamma`	`price`	appelant
`opt::NMParams`, `Vertex`, `Simplex`	paramètres et sommets du simplexe	—	**personne** (code mort)

Note de conception : `OptionQuote` porte `ExerciseType` (Européen/Américain) mais **aucun consommateur ne le lit** aujourd'hui. Le champ existe pour le futur fit américain.

2.3 Catalogue des fonctions

Format : **nom — ce qu'elle fait en une ligne** *(appelée par)* → appelle.

pricing/blackscholes.cpp

Fonction	Rôle (une ligne)	Appelée par
`normalCDF(x)`	N(x), loi normale standard (via `erfc`)	`callPrice`, `putPrice`
`normalPDF(x)`	densité normale standard ϕ(x)	`vegaForward`
`d1(BSParams)`	`[ln(S/K)+(r−q+σ²/2)T]/(σ√T)`	`d2`, `callPrice`, `putPrice`, `vega`
`d2(BSParams)`	`d1 − σ√T`	`callPrice`, `putPrice`
`callPrice(BSParams)`	prix Black-Scholes d'un call européen spot	`impliedVol`, main
`putPrice(BSParams)`	idem pour un put	`impliedVol`, main
`callPriceForward(F,K,T,df,σ)`	prix Black-76 d'un call (forward + facteur d'actualisation)	main (inversion maison), tests
`putPriceForward(...)`	idem pour le put	tests
`vegaForward(...)`	∂C/∂σ en forme forward	main, tests

calibration/ImpliedVol.cpp

Fonction	Rôle	Appelée par
`vega(BSParams)` *(anonyme)*	vega d'une option spot	`Newton`
`Newton(f, df, σ₀, tol, iter)` *(anonyme)*	Newton-Raphson générique sur `f(σ)=0`	`impliedVol`
`impliedVol(prix, params, type, σ_min, σ_max, tol, iter)`	prix → σ, avec un `variant` valeur/erreur	main, runner, tests

calibration/SSVI.cpp

Fonction	Rôle	Appelée par
`theta(T,p)`	variance ATM `θ = ν·T`	`totalVariance`, dérivées
`phi(th,p)`	pente de l'aile `η/(θ^γ(1+θ)^(1−γ))`	`totalVariance`, dérivées
`totalVariance(k,T,p)`	`w(k,T)` — la surface	`impliedVolSSVI`, `LocalVol`, diagnostics
`impliedVolSSVI(k,T,p)`	`σ = √(w/T)`	objectif, diagnostics, tests
`dw_dk`, `d2w_dk2`, `dw_dT`	dérivées analytiques de `w`	`LocalVol`
`isArbitrageFree(p)`	les 5 conditions (Thm 4.1 + 4.2 + bornes)	`calibrateSSVI` (certification finale)
`phiSqSupFactor(γ)` *(anonyme)*	`c(γ) = a^a/(1+a)^(1+a)`, `a = 1−2γ`	`etaMax`, `isArbitrageFree`
`etaMax(γ,ρ)` *(anonyme)*	`min(4/(1+|ρ|), 2/√(c(γ)(1+|ρ|)))`	`fromOptimiserVars`, point de départ
`fromOptimiserVars(x)` *(anonyme)*	`(ρ,u,γ,ν) → (ρ, u·η_max, γ, ν)`	objectif, sortie
`objective(x, grad, data)` *(anonyme)*	somme des carrés des écarts d'IV	NLopt
`calibrateSSVI(k_grid, T_grid, iv_matrix)`	l'optimisation complète, retourne un `variant`	main, runner

calibration/LocalVol.cpp et FitDiagnostics.cpp

Fonction	Rôle	Appelée par
`localVol(k,T,p)`	Dupire : σ_loc en un point	`localVolGrid`, main
`localVolGrid(k_grid,T_grid,p)`	la grille complète	main
`etaMaxBoundLocal`, `phiSqSupFactorLocal` *(anonymes)*	**duplication** de la logique de SSVI.cpp	`diagnoseSSVIFit`
`diagnoseSSVIFit(...)`	RMSE global/par échéance + `eta_ratio`	runner, tests

pde/Grid.cpp, Tridiag.cpp, PDEPricer.cpp

Fonction	Rôle	Appelée par
`make_uniform_grid(K, x_min, x_max, N)`	grille uniforme en `x = ln(S/K)`	`price`
`assemble_cn(A, B, grid, σ_loc, r, q, τ, T, dt, θ)`	remplit les deux tridiagonales du schéma Crank-Nicolson	`price`
`thomas_solve(A, rhs, out)`	résout le système tridiagonal (nœuds intérieurs)	`price`, tests
`apply_boundary(...)` *(statique)*	conditions de Dirichlet + correction du second membre	`price`
`price(type, style, S0, K, T, σ_loc, r, q, params)`	la boucle en temps complète, retourne prix + greeks	main
`price(type, style, S0, K, T, σ, r, q, params)`	surcharge à vol/taux constants	main, tests

data/

Fonction	Rôle	Appelée par
`RateCurve::fromCSV`	lit `(T, r)` et construit la courbe	`MarketData::load`
`RateCurve::rate(T)`	interpolation **linéaire** des taux, plate au-delà des bornes	`discount`, `forward`
`RateCurve::discount(T)`	`e^(−r(T)·T)`	`pvDividends`, `forwardDiscrete`
`RateCurve::forward(T1,T2)`	taux forward simple entre deux dates	(inutilisé)
`DivCurve::fromCSV`	lit `(date, montant)` → `(T années, α)`	`MarketData::load`
`DivCurve::pvDividends(t,T,curve)`	somme actualisée des dividendes de `]t,T]`	`forwardDiscrete`
`DivCurve::inInterval(t1,t2)`	liste des dividendes dans l'intervalle	(futur : sauts dans la PDE)
`forwardDiscrete(S,T,rates,divs)`	`(S − PV(div)) / D(T)`	main
`parseDate`, `yearsBetween`	dates `AAAA-MM-JJ` → années (base 365,25)	`DivCurve::fromCSV`
`MarketData::load(folder, today)`	lit `spot.txt`, `rates.csv`, `divs.csv`, `quotes.csv`	main, runner
`loadQuotes` *(statique)*	parse le CSV de cotations, ignore les lignes cassées	`MarketData::load`

opt/NelderMead.cpp

Fonction	Rôle	Appelée par
`minimize(f, x0, params)`	Nelder-Mead maison, **incomplet** (critère d'arrêt WIP)	**personne**

2.4 Les deux graphes d'appel qui comptent

(a) Calibration — c'est le chemin chaud, exécuté des milliers de fois par l'optimiseur.

```
main
 └─ MarketData::load
     ├─ loadQuotes            (parse CSV)
     ├─ RateCurve::fromCSV
     └─ DivCurve::fromCSV → yearsBetween
 └─ impliedVol (par cotation)
     └─ callPrice / putPrice → d1 → normalCDF ; d2
 └─ calibrateSSVI                      ← 1 appel
     └─ NLopt BOBYQA
         └─ objective  (≈ 10 000 appels)
             └─ fromOptimiserVars → etaMax → phiSqSupFactor
             └─ impliedVolSSVI → totalVariance → theta, phi
 └─ diagnoseSSVIFit                    ← 1 appel
     └─ impliedVolSSVI × (nT × nK)
     └─ etaMaxBoundLocal
```

Coût : `objective` est appelée jusqu'à 10 000 fois ; chaque appel balaie `nT × nK` points (par exemple 20 × 11 = 220). Soit ~2,2 M d'évaluations de `w`. C'est pour cela que `theta`, `phi` et `totalVariance` sont `noexcept` et sans allocation : elles sont dans la boucle la plus chaude du projet.

(b) Pricing par PDE

```
main
 └─ pde::price
     ├─ make_uniform_grid           (1 fois)
     ├─ σ_loc(S0, 0)                (pour choisir la largeur de grille)
     └─ boucle n = 0..M−1
         ├─ assemble_cn             (recalcule les coefficients : σ_loc par nœud)
         ├─ apply_boundary          (Dirichlet + correction du rhs)
         ├─ thomas_solve            (O(N))
         └─ max(V, intrinsèque)     (si américain)
```

2.5 Le contrat d'entrée du moteur (aujourd'hui)

Le moteur ne connaît pas la base. Il attend un **dossier** :

```
data/csv/<TICKER>/
├── quotes.csv     K,T,isCall,exerciseType,bid,ask,volume,openInterest
├── spot.txt       un seul nombre
├── rates.csv      T,r          (optionnel → plat 0 %)
└── divs.csv       date,montant (optionnel → aucun dividende)
```

Ce format est **un héritage de la chaîne CBOE** (l'exporteur Python qui l'écrivait a été supprimé avec la migration `004_retrait_cboe.sql`). Il reste un fichier SPX figé du 12 septembre dans `data/csv/SPX/`, que `calibrate_spx_minimal` sait encore lire — c'est ce qui permet au moteur de tourner, mais sur des données mortes.

C'est exactement le point de couture à refaire pour brancher Deribit (partie 5, étape 4).

---

3. N2 — Le moteur C++ en détail, fonction par fonction

Cette partie se lit avec le code ouvert à côté. Pour chaque fonction : **but**, **maths**, **entrées/sorties**, **pièges**, et ce qu'il faudrait changer.

3.1 pricing/blackscholes.cpp — les primitives

`normalCDF(x)` et `normalPDF(x)`

```
N(x) = ½·erfc(−x/√2)          φ(x) = e^(−x²/2)/√(2π)
```

`erfc` est utilisée plutôt que `0.5*(1+erf(x))` parce qu'elle garde de la précision dans l'aile gauche loin de zéro : pour `x = −8`, `1 + erf(x)` souffre d'annulation catastrophique, `erfc` non. Détail qui compte quand on inverse des IV d'ailes.

`d1`, `d2`

```
d₁ = [ln(S/K) + (r − q + ½σ²)T] / (σ√T)
d₂ = d₁ − σ√T
```

Piège mesuré : à `T = 0` et `S = K`, on calcule `0/0` → NaN. Le test `zero_or_near_zero_maturity_policy_is_explicit` documente le cas `T → 0` hors ATM (qui converge vers l'intrinsèque) mais **le cas ATM à T=0 n'a pas de politique** : le code retourne NaN. À trancher explicitement (retourner l'intrinsèque, ou refuser l'appel).

`callPrice`, `putPrice`

```
C = S e^(−qT) N(d₁) − K e^(−rT) N(d₂)
P = K e^(−rT) N(−d₂) − S e^(−qT) N(−d₁)
```

Forme **spot**, avec taux et dividende continus. Vérifiée par la parité dans les tests (`C − P = S e^(−qT) − K e^(−rT)` à 1e−12).

`callPriceForward`, `putPriceForward`, `vegaForward`

```
d₁ = [ln(F/K) + ½σ²T] / (σ√T),   d₂ = d₁ − σ√T
C = D·[F N(d₁) − K N(d₂)]
P = D·[K N(−d₂) − F N(−d₁)]
vega = D·F·√T·φ(d₁)
```

C'est la forme **à utiliser** pour tout ce qui vient du marché : elle ne demande que `(F, D)`, qui sont observables, et évite d'avoir à modéliser `r` et `q` séparément. Aujourd'hui, seule la version spot est utilisée par `impliedVol` ; `main.cpp` refait son propre Newton sur la version forward (voir §3.8, dette D3).

Astuce d'interopérabilité (utile si tu ne veux pas toucher à l'API) : passer en forme forward dans l'API spot en posant `S = F` et `r = q = −ln(D)/T`. Alors `S e^(−qT) = F·D` et `K e^(−rT) = K·D`, ce qui est exactement Black-76.

3.2 calibration/ImpliedVol.cpp — inverser le prix

Le problème

On cherche σ tel que `Prix_BS(σ) = Prix_marché`. La fonction `σ ↦ Prix(σ)` est **strictement croissante** (vega > 0) : il y a au plus une racine, et un encadrement suffit à la garantir.

Ce que fait le code aujourd'hui

```
Newton(f, df, σ₀, tol, maxIter):
    σ = σ₀ = (σ_min + σ_max)/2          // ici 2,5 par défaut
    répéter maxIter fois:
        si |f(σ)| < tol : retourner σ
        si |f'(σ)| < 1e−10 : échec (DidNotConverge)
        σ ← σ − f(σ)/f'(σ)
        si σ ≤ 0 : échec
    échec
```

Trois défauts, par ordre de gravité :

1. **Aucun encadrement.** Le pas de Newton peut sortir de `[σ_min, σ_max]` ; rien ne l'en empêche. En aile, vega → 0 et le pas explose : on converge vers une racine fausse ou on échoue. Départ à 2,5 (250 % de vol !) : pour une option à 30 % de vol, on traverse des régions où `d₁` est énorme.
2. **`NoArbitrageViolation` n'est jamais retourné.** Le type d'erreur existe, le code ne le produit jamais : un prix sous l'intrinsèque ou au-dessus de `S e^(−qT)` est simplement traité comme un échec de convergence. On perd l'information la plus utile (c'est une **donnée** fausse, pas un **solveur** qui patine).
3. **Le paramètre `tol` est une tolérance en prix**, pas en vol. Avec des prix SPX à 4 000, `tol = 1e−8` est déjà à la limite de la précision relative du double.

Le correctif (écrit et validé, cf. `docs/phase1_completion_workbook.md`)

Newton **protégé par bissection** : on maintient un intervalle `[lo, hi]` qui contient la racine, on le rétrécit à chaque évaluation avec le signe de `f`, et on n'accepte un pas de Newton que s'il reste **strictement** dans l'intervalle — sinon on bissecte.

```
impliedVol(prix, p, type, σ_lo, σ_hi, tol, iter):
    valider (T>0, S>0, K>0, 0<σ_lo<σ_hi, tol>0, iter>0, prix fini)   sinon InvalidInput
    [L, U] = bornes sans arbitrage du type
    si non (L < prix < U) : NoArbitrageViolation
    si f(σ_lo) > 0 ou f(σ_hi) < 0 : DidNotConverge      // racine hors bracket
    σ = (σ_lo + σ_hi)/2
    répéter:
        f = Prix(σ) − prix ; si |f| < tol : retourner σ
        si f < 0 : σ_lo = σ sinon σ_hi = σ               // rétrécit le bracket
        pas = −f/vega
        σ ← (σ_lo < σ + pas < σ_hi) ? σ + pas : (σ_lo+σ_hi)/2
    DidNotConverge
```

Bornes sans arbitrage utilisées (européen, strictes) :

```
call : max(S e^(−qT) − K e^(−rT), 0) < C < S e^(−qT)
put  : max(K e^(−rT) − S e^(−qT), 0) < P < K e^(−rT)
```

Deux pièges de test appris en écrivant la solution :

- une option ITM profonde à `T` court a une valeur temps < 1e–8 : l'information de vol a disparu dans l'arrondi du double. Il faut **sauter** ces points (tester la valeur temps, pas le prix) ;
- loin dans l'aile, comparer en **prix** (reprice à 1e–8) et non en σ : σ y est mal conditionnée.

3.3 calibration/SSVI.cpp — le cœur du modèle

`theta(T, p)`

```
θ(T) = ν·T
```

C'est la variance totale ATM. Conséquence directe : `σ_ATM = √ν`, **constante en maturité**. Toute la structure par terme ATM du marché est donc forcée d'être plate. C'est le choix de modélisation le plus lourd du projet (voir §5.6 et §6.7).

`phi(th, p)`

```
φ(θ) = η / [ θ^γ (1+θ)^(1−γ) ]
```

Interprétation : φ est la pente du smile en k=0, à un facteur ½ρ près. Elle décroît en `θ` (c'est-à-dire en maturité) comme `θ^(−γ)` pour `θ` petit, et comme `θ^(−1)` pour `θ` grand. Le paramètre `γ` règle **à quelle vitesse** la pente s'aplatit avec la maturité.

`totalVariance(k, T, p)`

```
w(k,T) = θ/2 · [ 1 + ρφk + √((φk + ρ)² + 1 − ρ²) ]
```

Trois propriétés utiles pour l'intuition :

- en `k = 0` : `w = θ/2 · [1 + √(ρ² + 1 − ρ²)] = θ`. Donc `w(0,T) = θ(T)` : cohérent ;
- quand `|k| → ∞`, `w ≈ θ/2·[(1+ρ)φk + …]` : asymptotes **linéaires** en k, pentes `θφ(1±ρ)/2`. C'est ce qui borne les ailes et rend le modèle docile ;
- c'est la seule forme SVI (avec celle de Gatheral) qui garantit l'absence d'arbitrage papillon **pour tout k** sous des conditions simples sur θ et φ. C'est là tout l'intérêt de SSVI par rapport à un SVI par tranche.

Les dérivées analytiques

`dw_dk`, `d2w_dk2`, `dw_dT` sont codées à la main. Elles servent à Dupire et évitent des différences finies bruitées. Vérification rapide de `dw_dk` :

```
u = φk + ρ,  D = √(u² + 1 − ρ²)
w = θ/2·(1 + ρφk + D)
w_k = θ/2·(ρφ + (φu)/D)
w_kk = θ/2 · φ²(1−ρ²)/D³
```

La dernière est toujours ≥ 0 : la surface est **convexe en k**, ce qui est la version locale de l'absence d'arbitrage papillon.

Pour `dw_dT`, il faut la dérivée de ϕ en θ :

```
dφ/dθ = −φ·(γ/θ + (1−γ)/(1+θ))
w_T = ν · [ ½(1 + ρφk + D) + θ½·(ρφ'k + φ'k·u/D) ]
```

C'est exactement ce qu'implémente `dw_dT`.

`isArbitrageFree(p)`

Cinq tests, dans l'ordre du code :

1. `|ρ| < 1` ;
2. `η(1+|ρ|) < 4` — Thm 4.2, condition 1 ;
3. `η²·c(γ)·(1+|ρ|) ≤ 4` — Thm 4.2, condition 2 (la contraignante) ;
4. `ν > 0` — Thm 4.1(a), θ croissant ;
5. `0 < γ ≤ 0,5` — nécessaire pour que le sup de `θφ²` soit fini.

`phiSqSupFactor(γ)` et `etaMax(γ, ρ)`

Avec `a = 1 − 2γ` :

```
c(γ) = sup_{θ≥0} θφ²/η² = a^a / (1+a)^(1+a)     (limite 1 en γ = ½)
η_max(γ,ρ) = min( 4/(1+|ρ|),  2/√(c(γ)(1+|ρ|)) )
```

Démonstration : on maximise `θ^(1−2γ)(1+θ)^(2γ−2)`, dont la dérivée s'annule en `θ* = 1−2γ`.

Le point subtil : pour `γ > ½`, l'exposant `1−2γ` est négatif, `θ^(1−2γ)` diverge quand `θ → 0`, donc le sup est infini et la condition 2 ne peut jamais tenir. D'où le `return infinity()` explicite — sans quoi `std::pow` d'une base négative avec un exposant fractionnaire rendrait un NaN, et **un NaN rend toutes les comparaisons fausses en silence**.

`fromOptimiserVars(x)`

```
x = (ρ, u, γ, ν)  →  (ρ, u·η_max(γ,ρ), γ, ν)
```

C'est le reparamétrage qui transforme les conditions du Thm 4.2 en **borne de boîte** `u ≤ 1−1e−4`. Raison technique : BOBYQA (NLopt) **refuse les contraintes non linéaires** ; il n'accepte que des bornes. La marge `1e−4` évite qu'un arrondi fasse échouer la certification finale.

`objective(x, grad, data)`

```
L(x) = Σ_i Σ_j [ σ_SSVI(k_j, T_i ; x) − σ_marché(i,j) ]²
```

Remarques de conception, à savoir défendre :

- c'est un **moindres carrés non pondéré** : chaque nœud de la grille commune pèse pareil. Une aile à vega quasi nulle (erreur de prix minuscule) compte autant que l'ATM. Pour du pricing c'est discutable ; pour de la **représentation** de surface c'est défendable ;
- c'est mesuré en **vol**, pas en variance totale ni en prix ;
- il n'y a **aucune régularisation** : rien n'empêche les paramètres de sauter d'un snapshot à l'autre. Pour des séries temporelles (le but à terme), il faudra un warm start et/ou une pénalité de saut.

`calibrateSSVI(k_grid, T_grid, iv_market)`

```
vérifier dimensions (sinon InvalidInput)
opt = NLopt LN_BOBYQA, 4 dimensions
bornes : ρ∈[-0.99,0.99], u∈[0.01, 1-1e-4], γ∈[0.01,0.5], ν∈[0.001,1]
départ : (-0.5, 1/η_max(0.3,-0.5), 0.3, 0.04)
xtol_rel = 1e-8, maxeval = 10 000
optimiser ; en cas d'exception → DidNotConverge
certifier isArbitrageFree(params) sinon ArbitrageViolation
retourner params
```

Points à connaître par cœur :

- **BOBYQA est un optimiseur sans gradient, à région de confiance** : robuste en petite dimension (4), lent en grande. C'est le bon choix ici ;
- le point de départ met `u = 1/η_max(0.3,-0.5)`, c'est-à-dire **η = 1** — un η absolu, pas un ratio. C'est cohérent mais fragile : si `η_max` change beaucoup avec `γ`, le départ est loin ;
- le résultat est **certifié** par `isArbitrageFree` — mais cette certification est *suffisante*, pas nécessaire (voir §6.5).

3.4 calibration/LocalVol.cpp — Dupire

La formule

En coordonnées de variance totale, la volatilité locale de Dupire s'écrit (Gatheral) :

$$\sigma_{loc}^2(k,T) = \frac{w_T}{1 - \frac{k}{w} w_k + \frac{1}{4}\left(-\frac{1}{4} - \frac{1}{w} + \frac{k^2}{w^2}\right) w_k^2 + \frac{1}{2} w_{kk}}$$

C'est **exactement** ce qu'implémente `localVol`, avec `k` en log-moneyness forward.

Lecture de la formule

- Le numérateur `w_T` est le **rythme** auquel la variance totale s'accumule. C'est lui qui porte la structure par terme.
- Le dénominateur est une **densité** (au sens de Breeden-Litzenberger). S'il devient négatif, la densité est négative : il y a arbitrage papillon, et σ_{loc}^2 n'a pas de sens.
- Le terme $-\frac{1}{4} - \frac{1}{w}$ vient du passage de la coordonnée strike à la coordonnée log.

Les garde-fous du code

```
si dénominateur ≤ 0        → NaN
si w_T < 0                 → NaN
si w < 1e-6                → NaN
```

Le problème n'est pas là, il est en aval. Dans `Tridiag::assemble_cn` :

```
if (std::isnan(sigma) || sigma < 1e-4) sigma = 1e-4;
```

Une surface localement arbitrageuse ne lève donc **aucune erreur** : elle est pricée avec une vol de 0,01 %. Un pricing faux et silencieux, exactement le type de bug que le projet a déjà payé deux fois. À remplacer par une erreur remontée au runner.

3.5 pde/ — l'équation aux dérivées partielles

`make_uniform_grid(K, x_min, x_max, N)`

```
x_i = x_min + i·Δx,  Δx = (x_max − x_min)/N,  S_i = K·e^(x_i),  i = 0..N
```

Grille **uniforme en log-moneyness**, centrée sur le strike. `N+1` nœuds, `N` intervalles. Dans `price`, la largeur est `±5·σ_atm·√T` : ±5 écarts-types lognormaux, ce qui met les conditions aux bords assez loin pour ne pas polluer le prix.

`assemble_cn(A, B, grid, σ_loc, r, q, τ, T, dt, θ)`

On discrétise, pour `x = ln(S/K)` et `τ = T − t` :

```
∂V/∂τ = ½σ²(x,τ)·V_xx + (r − q − ½σ²)·V_x − r·V
```

Notations du code (`c = r − q − ½σ²`) :

```
α = ½σ²/Δx² − c/(2Δx)        (coefficient du nœud j−1)
β = −σ²/Δx² − r              (coefficient du nœud j)
γ_pde = ½σ²/Δx² + c/(2Δx)    (coefficient du nœud j+1)
```

puis, pour le schéma à θ (Crank-Nicolson si θ = ½) :

```
A.lower[j] = −θ·α        A.diag[j] = 1/Δt − θ·β        A.upper[j] = −θ·γ_pde
B.lower[j] = (1−θ)·α     B.diag[j] = 1/Δt + (1−θ)·β    B.upper[j] = (1−θ)·γ_pde
```

Vérification utile : `α + β + γ_pde = −r`. Donc sur une fonction constante `V ≡ 1`, l'opérateur rend `−r` : c'est l'actualisation. Si ce test échoue, il y a une erreur de signe.

Détail important : les coefficients dépendent de `σ_loc(S_j, t)` évaluée **nœud par nœud**. C'est ce qui fait de la PDE une vraie PDE à coefficients variables — et c'est aussi pourquoi le coût est `O(M·N)` évaluations de `σ_loc` par pricing.

`thomas_solve(A, rhs, out)`

Algorithme de Thomas (élimination de Gauss pour tridiagonale), sur les **nœuds intérieurs** `j = 1..N−1`. Les valeurs de bord `out[0]` et `out[N]` sont supposées déjà fixées (Dirichlet).

```
pour i = 1..N−2:  w = lower[j]/d[i−1] ;  d[i] −= w·upper[j−1] ;  r[i] −= w·r[i−1]
substitution arrière de i = N−2 jusqu'à 0
```

Trois pièges de ce type de code, tous présents dans les tests existants :

1. **conventions d'indices** : `out` est indexé sur les nœuds (`0..N`), les tableaux internes sur `0..N−2` . Un décalage d'un cran donne une solution fausse mais **plausible** ;
2. `A.lower[0]` et `A.upper[N]` doivent rester à 0 (pas de voisin) ;

3. **aucune détection de pivot nul** : si un `d[i]` devient 0, la division donne `inf` / `NaN` sans erreur. À ajouter si la PDE doit tourner en production.

Complexité `O(N)` par pas de temps — c'est tout l'intérêt par rapport à une résolution dense `O(N³)`.

`apply_boundary(type, K, τ, r_curve, q_curve, T, grid, A, V, rhs)`

Conditions de Dirichlet aux deux extrémités :

```
call : V[0] = 0,                              V[N] = max(S_N·e^(−qτ) − K·e^(−rτ), 0)
put  : V[0] = K·e^(−rτ),                      V[N] = 0
```

Puis la **correction dans le second membre**, indispensable puisqu'on ne résout pas les nœuds de bord :

```
rhs[1]   −= A.lower[1]·V[0]
rhs[N−1] −= A.upper[N−1]·V[N]
```

Oublier cette correction est l'erreur classique : le prix est faux de quelques pourcents, et plus on raffine la grille, plus l'erreur **augmente** au lieu de diminuer (signature diagnostique).

`price(type, style, S0, K, T, σ_loc, r_curve, q_curve, params)`

```
σ_atm = σ_loc(S0, 0)
grille = make_uniform_grid(K, −5σ_atm√T, +5σ_atm√T, N)
Δt = T/M
V = payoff(S, K)                                  // τ = 0
pour n = 0..M−1:
    τ = n·Δt
    assemble_cn(A, B, …, τ, T, Δt, θ)
    rhs = B·V                                     // nœuds intérieurs
    apply_boundary(…, τ+Δt, …, V, rhs)            // Dirichlet + correction
    V_new = thomas_solve(A, rhs)                  // intérieurs
    si américain: V_new[j] = max(V_new[j], intrinsèque[j])
    V = V_new
prix  = interpolation linéaire de V en x0 = ln(S0/K)
delta = (V[j+1] − V[j−1]) / (S[j+1] − S[j−1])
gamma = (V[j+1] − 2V[j] + V[j−1]) / (Δx·S_j)²
```

Points de vigilance :

- **`τ` vs `τ + Δt`** : le code assemble avec `τ` (pour les coefficients) et applique les bords avec `τ + Δt` (l'instant d'arrivée). C'est la bonne convention, mais c'est le genre de détail qui se casse silencieusement à la première refonte ;
- **Crank-Nicolson « ringue »** au premier pas quand le payoff n'est pas lisse (coude au strike). La parade standard est un premier pas implicite (Rannacher, θ=1 sur les deux premiers demi-pas). Absent ici : c'est une source d'oscillations sur les greeks, pas sur le prix ;
- **grille uniforme en x** : précision concentrée près du strike, ce qui est le bon choix pour un payoff à coude unique, mais coûteux en `N` (400 nœuds pour 1e−4 de précision relative) ;
- **exercice anticipé** : `max(V, intrinsèque)` appliqué après chaque pas est la méthode explicite (« projection »), correcte à l'ordre 1 en `Δt`. Standard et suffisant pour un premier jet ; les méthodes de Brennan-Schwartz ou PSOR convergent plus vite mais sont plus lourdes.

Survivances à traiter plus tard

Le fichier contient des marqueurs `AFFINE DIV (futur)` : le pricer suppose des dividendes **continus** (via `q_curve`). Pour des dividendes discrets (actions, ou crypto si un jour il y a des distributions), il faut, à chaque pas de temps où tombe une ex-date, décaler la grille : `V(x) ← V(x − ln(1 − β)) − α` avant d'appliquer l'exercice. C'est écrit en TODO, pas en code.

3.6 data/ — le contrat d'entrée et ses défauts

`RateCurve`

```
rate(T)  = interpolation LINÉAIRE des taux entre tenors, plate au-delà
discount(T) = exp(−rate(T)·T)
forward(T1,T2) = (rate(T2)T2 − rate(T1)T1)/(T2−T1)
```

Deux défauts de modélisation, à connaître :

1. **interpolation linéaire sur les taux zéro, pas sur les log-discounts** : le forward implicite `f(T)` saute aux nœuds. Correctif standard : interpoler `ln D(T)` (monotone, forward continu).
2. **extrapolation plate** au-delà du dernier tenor : pour une maturité à 3 ans avec une courbe qui s'arrête à 1 an, on réutilise le taux 1 an. Acceptable en dessous de quelques années, dangereux pour du long terme.

`DivCurve` et le modèle affine

Chaque dividende porte `(T, α, β)` :

```
dividende à T  :  montant en cash α, plus une fraction β du spot
payoff ajusté  :  S → S(1−β_T) − α_T
```

- `pvDividends(t,T,curve)` ne somme que **`α` actualisé** — la partie `β` (proportionnelle) n'est pas dans la PV parce qu'elle ne se calcule pas en cash : elle se comporte comme un rendement continu et doit entrer dans `q`.
- `forwardDiscrete(S,T,rates,divs) = (S − PV(div)) / D(T)` : c'est la formule correcte du forward avec dividendes discrets.
- `inInterval(t1,t2)` existe mais n'est **utilisé nulle part** : c'est le futur crochet pour les sauts de la PDE.

État réel du modèle affine : la partie `β` est implémentée à moitié. `DivCurve` la stocke, `pvDividends` l'ignore (correct), mais rien ne l'injecte dans `q_curve` de la PDE. Le commentaire `AFFINE DIV (futur)` dans `PDEPricer.cpp` le dit honnêtement.

`MarketData::load(folder, today)`

```
spot  ← spot.txt                        (optionnel, 0 si absent)
rates ← rates.csv  sinon plat 0 %       (optionnel)
divs  ← divs.csv   sinon aucun          (optionnel)
calls/puts ← quotes.csv                 (OBLIGATOIRE)
```

Défaut constaté : `loadQuotes` fait `catch (...) { continue; }`. Toute ligne invalide est silencieusement jetée, et `std::stod("12abc")` renvoie 12 sans erreur. Une évolution de schéma côté Python passerait inaperçue. Le correctif (compteur de lignes rejetées + parsing strict) est dans le carnet de phase 1.

Défaut de conception plus profond : le loader **mélange I/O, parsing et valeurs par défaut**. Pour brancher la base Deribit, il faudra séparer (a) l'acquisition (DB ou fichier), (b) le parsing, (c) les hypothèses (taux plat, forward). C'est l'objet de la partie 5.

3.7 opt/NelderMead.cpp — à laisser dormir

Le code est un Nelder-Mead incomplet : le critère d'arrêt est en commentaire marqué `WIP`, et la signature définie ne correspond pas à celle déclarée. Il ne sert à rien aujourd'hui.

Recommandation : ne **pas** le terminer maintenant. Un optimiseur maison se valide sur des objectifs jouets avant d'être branché ; tant que SSVI est calibré par BOBYQA et que le fit n'est pas raccordé aux données réelles, comparer les deux optimiseurs ne prouve rien.

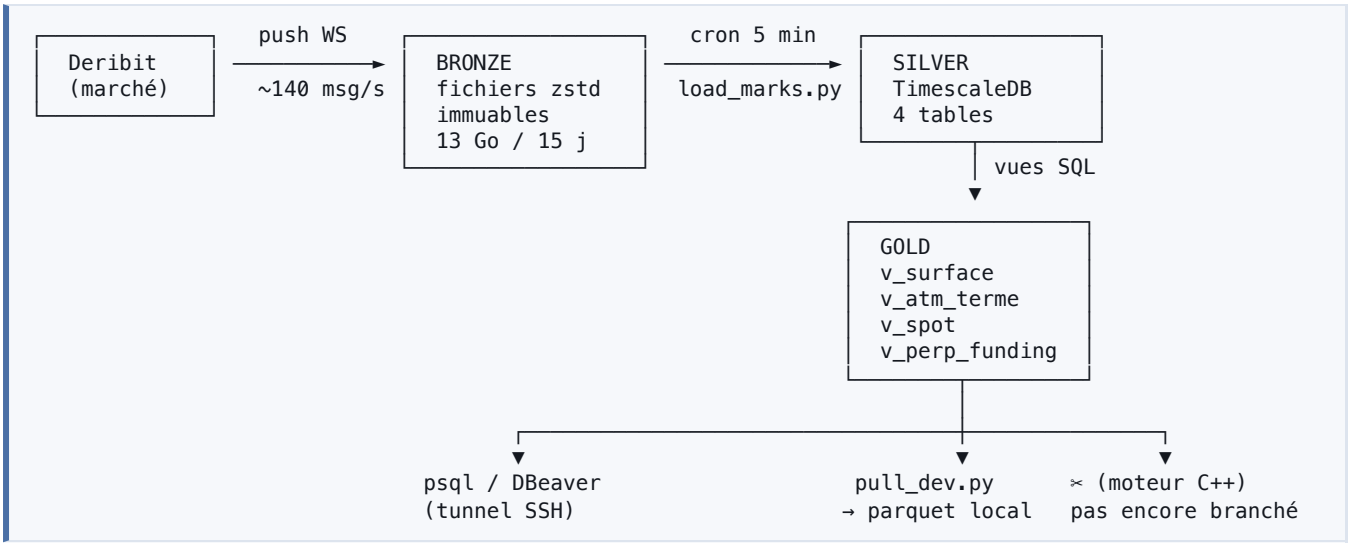
3.8 Dette technique relevée pendant la revue

#	Où	Problème	Sévérité	Correctif
D1	`ImpliedVol.cpp`	Newton pur, sans bracket ; `NoArbitrageViolation` jamais retourné	**haute**	Newton protégé (§3.2)
D2	`Tridiag.cpp`	`NaN` de σ_loc écrasé en `1e−4` sans erreur	**haute**	propager une erreur
D3	`src/main.cpp`	deuxième inversion d'IV, faite à la main sur `callPriceForward`, sans tolérance ni gestion d'échec	moyenne	supprimer, passer par la bibliothèque
D4	`opt/NelderMead`	header/cpp incohérents, code mort	faible	supprimer ou réparer
D5	`MarketData::loadQuotes`	`catch(...)`, lignes perdues en silence	moyenne	compteur + parsing strict
D6	`blackscholes`	ATM à `T=0` → NaN, pas de politique	faible	décider et tester
D7	`RateCurve`	interpolation linéaire sur les taux, extrapolation plate	moyenne	interpoler `ln D(T)`
D8	`PDEPricer`	pas de Rannacher, pas de détection de pivot nul	moyenne	θ=1 sur 2 demi-pas, garde sur `d[i]`
D9	`tests/`	`test_ssvi_synthetic` retourne 0 sans rien tester ; `test_implied_vol`, `test_market_data`, `test_pde_pricer` n'ont pas de `main`	**haute**	le premier est le plus dangereux : il **passe**
D10	`FitDiagnostics.cpp`	duplication de `etaMax` (risque de divergence avec SSVI.cpp)	faible	exposer un helper public

---

4. Le pipeline de données (Python + SQL)

4.1 N0 — La forme du pipeline



Trois décisions structurent tout le reste :

1. **Le brut n'entre jamais en base.** Un message par changement de carnet → des centaines de millions de lignes par jour. En fichiers zstd, c'est 13 Go pour 15 jours, rejouable à l'identique, déplaçable, sauvegardable.
2. **La base ne contient que ce qu'on interroge** : des états **échantillonnés** en tranches de 5 minutes, une ligne par instrument et par tranche.
3. **Le temps qui compte est celui du fournisseur** (`observed_at`), jamais l'heure d'exécution. Sans cela, rejouer un fichier ancien écrirait des lignes « datées d'aujourd'hui » et la reproductibilité des fits disparaîtrait.

4.2 N1 — Volume et rythme, mesurés

Grand	Valeur mesurée (28 oct. 2026)
Bronze sur disque	13 Go pour 15 jours ≈ 0,9 Go/jour
Instruments en dimension	3 370 (1 686 BTC, 1 684 ETH)
Échéances couvertes	27 par devise, jusqu'au 24 sept. 2027
Marques d'options en base	1 038 072
Tranches de 5 min distinctes	4 453
Ticks perpétuel / points DVOL	8 905 / 8 902
Retard base vs marché	borné par tranche + cron ≈ 10 min
Coût disque de la base	quelques centaines de Mo (compression Timescale après 7 j)

Un point de rythme à retenir : **bronze → base est un sondage** (cron toutes les 5 min), alors que **Deribit → bronze est un push** (WebSocket). Ce sont deux maillons de nature différente, et les confondre est l'erreur d'architecture la plus fréquente sur ce genre de projet.

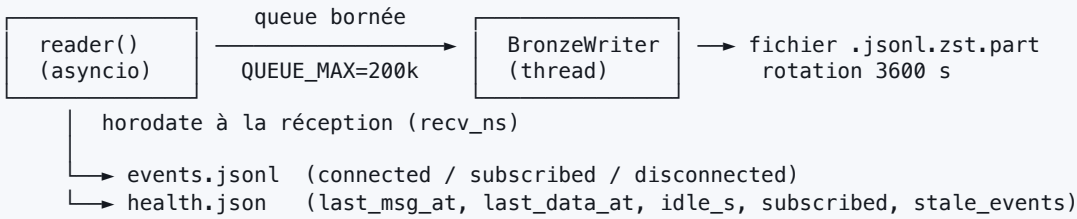
4.3 N1 — Catalogue des modules Python

Module	Rôle	Dépend de
`python/collect/deribit_ws.py`	collecteur WebSocket → bronze	`websockets`, `zstandard`, `names`
`python/collect/names.py`	parsing des noms d'instruments (`BTC-24SEP27-104000-P`)	stdlib seule (volontairement)
`python/collect/analyze_bronze.py`	rapport qualité d'un run bronze (volume, latence, trous)	`zstandard`
`python/jobs/load_marks.py`	bronze → silver	`zstandard`, `names`, `db.connection`
`python/db/connection.py`	ouverture de connexion psycopg (env `PG*`)	`psycopg`
`python/db/migrate.py`	runner de migrations SQL, avec registre sha256	`psycopg`
`python/io/atomic.py`	écriture atomique de fichiers (tmp + `os.replace`)	stdlib
`scripts/pull_dev.py`	extraction base → parquet local + manifeste	`psycopg`, `pandas`
`scripts/plot_surface.py`	trace le smile et la structure par terme depuis les vues	`pandas`, `matplotlib`
`scripts/db-tunnel.sh`	tunnel SSH `15432 → mini:5432`	`ssh`
`scripts/macmini_bootstrap.sh`	installation complète du mini	`brew`, `docker`
`ops/*.sh`	cron : garde-fou, alertes, chargement, backups, restauration	shell

Pourquoi `names.py` est séparé : le collecteur importe `websockets` ; les jobs de chargement ne doivent pas avoir besoin d'une dépendance réseau pour lire un nom d'instrument. La séparation est une contrainte de déploiement, pas un caprice.

4.4 N2 — `deribit_ws.py` , le collecteur

Architecture interne (le point qui compte)



Le lecteur **ne fait que** recevoir, horodater, mettre en file. Aucun parsing, aucune écriture disque. C'est ce qui évite la contre-pression : si le disque ralentit, la file absorbe, et le socket continue d'être vidé.

Les canaux souscrits et ce qu'ils portent

Canal	Contenu	Volumétrie
`quote.{instrument}`	meilleur bid/ask + tailles, toute la chaîne (~1 700 options/devise)	~157 msg/s
`markprice.options.{btc,eth}_usd`	**mark_iv + mark_price de TOUTE la chaîne** (~1 msg/s/devise)	faible
`ticker.{instrument}.100ms`	greeks, open interest, bid/ask IV — bande ATM seulement (20 instruments/devise)	~0,3–1 msg/s/instrument
`ticker.{BTC,ETH}-PERPETUAL.100ms`	funding, base, open interest, index	faible
`trades.option.*.100ms`, `trades.{PERP}.100ms`	prints	variable
`deribit_volatility_index.{btc,eth}_usd`	DVOL	faible
`deribit_price_index`, `estimated_expiration_price`, `deribit_price_ranking`	références et composition d'indice	faible

Décision importante : `ticker.*` sur toute la chaîne coûterait ~60 Go/jour. On le limite à une **bande ATM** (`select_ticker_subset`), recalculée à chaque session : les 5 échéances les plus proches, moitié calls moitié puts autour du spot. Conséquence : les greeks et le bid/ask d'IV n'existent que pour cette bande ; ailleurs, ils sont NULL — et ce NULL est une information (l'instrument n'était pas dans la bande).

Les mécanismes de survie (chacun correspond à un incident vécu)

Mécanisme	Ce qu'il empêche	Constante
`set_heartbeat` 10 s + timeout 2×	socket mort sans erreur	`HEARTBEAT_S = 10`
watchdog sur la dernière DONNÉE (pas sur le heartbeat)	« abonné mais muet » : le heartbeat continue d'arriver alors que les souscriptions ne livrent plus (panne réelle : 53 h)	`STALE_S = 300`
`open_timeout`	un `connect()` qui traîne bloque le lecteur	`OPEN_TIMEOUT_S = 15`
`recover_stale_parts`	un `kill -9` laisse un `.part` invisible pour tout ce qui lit `*.jsonl.zst` (122 962 lignes sauvées une fois)	`min_age_s = 60`
noms de fichiers uniques (`-1`, `-2`)	un redémarrage dans la même heure écrasait le fichier de la session précédente (8 526 lignes perdues)	`_unique`
`stop_grace_period: 30s` (compose)	SIGKILL pendant l'écriture	—
`restart: unless-stopped` + garde-fou cron	Colima qui ne revient pas au boot (9 h de marché perdues)	`*/5`

Ce tableau est, à lui seul, un bon sujet d'entretien : « quels sont les modes de défaillance d'un collecteur temps réel ? ».

Le format bronze

```
{"r": 1761062400123456789, "m": { ...message Deribit brut... }}
```

`r` = horodatage de **réception** en nanosecondes. Le message `m` est écrit **tel quel**, sans traitement. C'est ce qui rend le bronze rejouable : on peut changer d'avis sur la curation (parser, filtres, unités) et rejouer 15 jours de marché sans recollecter.

4.5 N2 — `load_marks.py`, de bronze à base

Les étapes

```
fichiers_a_lire(bronze, since_hours=3, tout)
   └─ sélection par NOM de fichier (date=…/HHMMSSZ…), sans ouvrir
   └─ inclut le fichier de l'heure en cours (`*.jsonl.zst.part`)
pour chaque fichier :
   extraire([fichier], bucket_s=300)  → Echantillon
       ├─ markprice.options.*  → mark_iv (×100 !), mark_price        (toute la chaîne)
       ├─ ticker.* (non perp)  → bid/ask IV, prix, OI, greeks        (bande ATM)
       ├─ ticker.*PERPETUAL    → index, funding, base, OI
       └─ deribit_volatility_index → DVOL
   charger(conn, echantillon)
       ├─ ancrer_instruments   (dimension SCD2)
       ├─ inserer option_mark / perp_tick / vol_index  (COPY + ON CONFLICT DO NOTHING)
       └─ fermer_expires       (valid_to = expiry + 8 h)
   conn.commit()                                  ← UNE transaction PAR FICHIER
```

Les trois subtilités qui font la valeur du job

1. **Le bucket de 5 minutes.** Chaque observation est ramenée au début de sa tranche (`bucket_of`). Toutes les sources qui parlent dans la même tranche pour le même instrument sont **fusionnées** dans une seule ligne (`fusionne_marque`), et la colonne `source` dit laquelle ou lesquelles ont contribué : `markprice`, `ticker`, ou `markprice+ticker`. C'est ce qui permet d'avoir IV et prix sur toute la chaîne, et greeks/bid-ask sur la bande.
2. **La dernière tranche d'un `.part` est écartée.** Une ligne déjà écrite n'est jamais réécrite (`ON CONFLICT DO NOTHING`). Si on écrivait une tranche encore en cours de remplissage, elle serait **figée sur ses premières secondes** pour toujours. On la laisse au passage suivant. Effet mesuré : retard de la base ramené de ~52 min à ~7 min.
3. **Une transaction par fichier, pas une pour le lot.** Mémoire (un rattrapage de 77 fichiers accumulerait 3 M lignes) et résilience (un échec au 76e ne perd pas les 75 précédents).

La dimension SCD2 (`ancrer_instruments`)

```
instrument(instrument_sk, provider, underlying, instrument_name, kind,
           expiry, strike, option_type, ...,
           valid_from, valid_to, recorded_at)
```

- `valid_from` = **première observation dans le bronze**, pas l'heure du chargement. Sinon, rejouer un bronze ancien créerait un instrument « né aujourd'hui mais expiré hier », ce que la contrainte de cohérence refuse.
- `valid_to` = `expiry + 8 h` (les options Deribit expirent à 08:00 UTC).
- Contrainte d'exclusion `btree_gist` : deux versions d'un même instrument ne peuvent pas se chevaucher. C'est ce qui a obligé à écrire une recherche préalable (« une version couvre-t-elle déjà cet instant ? ») avant d'insérer — leçon apprise au premier rattrapage complet.

L'écriture

```
CREATE TEMP TABLE tmp_load_<table> (LIKE silver.<table>) ON COMMIT DROP
COPY tmp_load_<table> FROM STDIN         ← dizaines de milliers de lignes en une fraction de seconde
INSERT INTO silver.<table> SELECT ... ON CONFLICT DO NOTHING
```

`COPY` puis `INSERT ... ON CONFLICT` : on garde la vitesse du `COPY` **et** l'idempotence de `ON CONFLICT`. Le nom de la table temporaire dépend de la table cible (sinon deux chargements dans la même transaction échouent sur un `DuplicateTable` — vécu).

4.6 N2 — `migrate.py`, le runner de migrations

Pourquoi il existe : `docker-entrypoint-initdb.d` ne s'exécute **qu'à la première initialisation** du volume. Donc jamais sur une base existante, et sans aucune trace de ce qui a été appliqué.

Règles appliquées par le runner :

- fichiers `sql/NNN_nom.sql`, numéro croissant, jamais réutilisé ;
- **sha256 enregistré** : retoucher une migration déjà appliquée fait échouer le runner (exit 3) **avant** toute écriture. On corrige par une migration suivante.
- une transaction par fichier, sauf `-- migrate:no-transaction` (nécessaire pour `CREATE INDEX CONCURRENTLY`) ;
- verrou consultatif : deux exécutions simultanées impossibles ;
- `--status`, `--dry-run`, `--baseline-through 001` (pour une base créée avant les migrations).

C'est ce qui a permis à `004_retrait_cboe.sql` de **supprimer** des tables sans jamais réécrire `001_schema.sql` : on ne réécrit pas l'histoire, on ajoute une étape.

4.7 N1 — Les tables et les vues

Objet	Nature	Clé / particularité
`silver.instrument`	dimension	SCD2, exclusion sur chevauchement de période
`silver.option_mark`	hypertable	`(instrument_sk, observed_at)`, compression > 7 j
`silver.perp_tick`	hypertable	idem, funding/base/OI
`silver.vol_index`	hypertable	`(underlying, observed_at)`
`gold.v_spot`	vue	dernier état du perpétuel par devise
`gold.v_surface`	vue	une ligne par option : `k_logm`, `t_annees`, `iv_pct`, prix
`gold.v_atm_terme`	vue	ATM par échéance (strike le plus proche du spot)
`gold.v_perp_funding`	vue	funding en bp/8 h et annualisé
`public.schema_migrations`	registre	hors couches, emplacement fixe

Trois pièges inscrits dans les commentaires SQL, et qui valent d'être connus :

1. `markprice.options` est un flux **incrémental** : une tranche de 5 min ne contient que ce qui a bougé. « La surface maintenant » se lit donc **par instrument** (dernière observation), pas par tranche. `v_surface` fait un `DISTINCT ON (instrument_sk) … ORDER BY observed_at DESC`.
2. Le spot doit venir du **perpétuel**, pas des colonnes de l'option : celles-ci ne sont remplies que par `ticker`, donc seulement dans la bande ATM. Sinon le « strike le plus proche du spot » retombe sur un strike arbitraire — mesuré : ATM à 92,86 % au lieu de 38,25 % sur BTC.
3. Les IV de Deribit sont publiées en **deux unités** selon le canal (fraction vs pourcentage) ; la colonne est normalisée en pourcentage et protégée par un `CHECK` qui **refuse** une valeur dans `]0,1[` (la signature du bug).

4.8 N1 — Exploitation

```
*/5  mini-stack-guard.sh     Colima + stack, si besoin   (pas de session graphique sur le mini)
*/10 mini-alerte.sh          ntfy : sauvegarde, collecte, base, disques
2-59/5 mini-load-marks.sh    bronze → base
5    mini-backup-launch.sh --bronze-only     bronze → disque externe
30 2 mini-backup-launch.sh                   bronze + pg_dump
0 4 * * 0 mini-verif-restauration.sh         restauration de contrôle hebdomadaire
```

Deux leçons d'exploitation déjà payées :

- **cron n'a pas l'autorisation macOS d'écrire sur un volume externe** (pas de session graphique pour l'accorder) : d'où `ssh localhost`, dont le processus a l'autorisation ;
- **modifier le collecteur exige un rebuild** : le code est dans l'image, pas monté. Un simple `up -d` recrée le conteneur avec l'ancien code et boucle sur des arguments inconnus.

4.9 N1 — Le poste de développement (MacBook)

```
scripts/db-tunnel.sh start      ssh -L 15432 → mini:5432   (start/stop/status, keepalive)
scripts/pull_dev.py --jours 2   → data/dev/marches.parquet, perpetuel.parquet,
                                  v_surface.parquet, manifeste.json
code .                          notebooks/01_premier_smile.ipynb  (hors ligne)
```

La base du mini est **en production** : on lit, on ne scanne pas tout, on ne calcule rien de lourd dessus. Les gros calculs se font en local sur le parquet. Le manifeste porte la requête exacte et le commit : un résultat peut toujours être ramené à ce qui l'a produit.

4.10 N1 — Tests Python

```
49 passed, 14 skipped   (les skips = tests d'intégration qui exigent une base)
```

Les tests couvrent : parsing des noms d'instruments, canaux souscrits, récupération des `.part`, watchdog, chargement idempotent, SCD2, runner de migrations (unitaires + intégration). C'est la partie la mieux testée du dépôt — à l'inverse du C++.

---

5. Modélisation : taux, dividendes, forward — et comment passer à la crypto

C'est la partie où les décisions coûtent le plus cher, parce qu'elles sont **silencieuses**. Un taux faux de 10 bp ne se voit pas ; un forward faux de 2 % déforme le skew et personne ne s'en aperçoit tant qu'on ne compare pas à un prix de marché.

5.1 Pourquoi ces objets existent

Le modèle SSVI ne parle ni de taux ni de dividendes : il travaille sur `(k, T, σ)`. Mais le passage **du prix au smile** exige trois choses :

1. **un forward `F(T)`** : c'est l'origine de l'axe des strikes (`k = ln(K/F)`). Un forward faux translate tout le smile et, s'il est faux de façon non constante en `T`, il **déforme le skew**.
2. **un facteur d'actualisation `D(T)`** : c'est ce qui transforme un prix coté en espérance risque-neutre. Faux, il déplace toutes les IV, différemment selon les strikes (les calls ITM pèsent `D·F`, les puts `D·K`).
3. **une convention de temps** : `T` en années, échéance à une heure précise. Un `T` faux de 8 h sur une maturité d'une semaine, c'est 5 % d'erreur sur `T`, donc ~2,5 % sur σ.

Les deux premiers sont reliés :

```
Parité call-put :   C − P = D·(F − K)
```

Cette identité est **sans modèle** : elle ne suppose ni Black-Scholes, ni vol constante. C'est pourquoi c'est le meilleur outil pour déterminer `(F, D)` à partir du marché — et le seul qui ne demande aucune hypothèse sur les taux.

5.2 Ce qui est implémenté aujourd'hui (actions / indice)

`RateCurve` — taux

```
struct RateCurve { vector<double> tenors, rates; };
rate(T)     = interpolation LINÉAIRE des taux zéro entre tenors ; PLATE hors bornes
discount(T) = exp(−rate(T)·T)
forward(T1,T2) = (rate(T2)·T2 − rate(T1)·T1) / (T2 − T1)
```

Trois remarques de modélisation :

- **interpoler des taux zéro linéairement** donne un forward implicite en escalier : `f(T)` saute à chaque nœud. Pour pricer correctement, il faut interpoler `ln D(T)` (typiquement linéaire ou en spline monotone), puis `rate(T) = −ln D(T)/T`. Différence négligeable à 3 mois, significative sur une courbe pentue à 5-10 ans ;
- **extrapolation plate** : une maturité au-delà du dernier tenor réutilise le dernier taux. Sur des options crypto à 1 an avec une courbe qui s'arrête à 3 mois, c'est une erreur pure ;
- `forward(T1,T2)` (taux forward simple) est écrit mais **jamais appelé** : le projet n'a pas encore de produit à coupons.

`DivCurve` — dividendes, modèle affine

Chaque dividende est `(T, α, β)` :

```
à la date T :  le porteur reçoit  α  en cash  +  une fraction β du spot
               ⇒  S_{T+} = S_{T−}(1 − β) − α
```

C'est le modèle « affine » (ou escrowed), plus général que le rendement continu `q` :

Modèle	Formule du forward	Quand il est correct
rendement continu `q`	`F = S e^((r−q)T)`	indice à dividendes lisses (SPX)
dividende discret cash	`F = (S − Σ α_i D(t_i)) / D(T)`	action avec dates connues
affine `(α, β)`	`F = (S(1−Σβ_i·…) − Σα_i D(t_i)) / D(T)`	cas général

Ce que le code fait réellement :

```
pvDividends(t,T,curve)  →  Σ α_i · D(t_i)   pour t < t_i ≤ T       ✔ la partie cash
forwardDiscrete(S,T)    →  (S − pvDividends(0,T)) / D(T)           ✔
inInterval(t1,t2)       →  liste des dividendes de l'intervalle    ✔ mais non utilisée
partie β (proportionnelle)                                         ✘ stockée, jamais consommée
```

Le modèle affine est donc implémenté à exactement 50 % : la partie cash est correcte, la partie proportionnelle est stockée et ignorée. Le commentaire `--- AFFINE DIV (futur)` dans `PDEPricer.cpp` marque les trois endroits à compléter :

1. remplacer `S0` par `S0* = S0 − PV(div cash)` dans la grille ;
2. ajouter `β/T` au `q_curve` (la partie proportionnelle est un rendement continu équivalent) ;
3. à chaque pas de temps contenant une ex-date, **décaler la grille** : `V(x) ← V(x − ln(1−β)) − α` avant d'appliquer l'exercice anticipé.

Le point 3 est celui qu'on oublie toujours : sans lui, le prix d'une option qui enjambe une ex-date est faux de l'ordre de `α`, et l'erreur ne diminue **pas** quand on raffine la grille.

Ce que fait le pricer aujourd'hui

`PDEPricer` reçoit `r_curve(t)` et `q_curve(t)` comme des **fonctions du temps** et les évalue à chaque pas. Il ne connaît ni `RateCurve` ni `DivCurve` : il voit des lambdas. C'est un bon découpage (le pricer est testable avec des taux plats), mais cela signifie que **la conversion courbe → fonctions est faite par l'appelant**, donc hors du périmètre testé.

5.3 Les conventions de temps (source d'erreurs sous-estimée)

Question	Choix actuel	Ce que ça implique
Base	365,25 jours (`yearsBetween` , `v_surface.t_annees`)	cohérent crypto et actions
Heure d'échéance	08:00 UTC (Deribit), 16:00 ET (SPX historique)	ne jamais mélanger deux places dans une même surface
Jour férié / week-end	ignoré (temps calendaire pur)	acceptable en crypto (24/7), **faux** pour une action
Act/365 vs Act/365F	365,25 partout	écart ~0,07 % sur `T` : négligeable mais doit être constant

Règle : **une seule convention par surface**, écrite dans le code et pas seulement dans la tête de celui qui l'a codée. Le mélange SPX (règlement à l'ouverture) et SPXW (règlement à la clôture) avait été explicitement écarté pour cette raison — c'est un précédent à réutiliser.

5.4 La réalité crypto : ce qui change tout

(a) Il n'y a pas de dividende, mais il y a un basis

Sur BTC/ETH, personne ne paie de dividende. En revanche :

- le **perpétuel** a un *funding* toutes les 8 h : c'est le coût de portage, réglé en continu ;
- les **futures datés** cotent un prix différent du spot : le *basis*. Il reflète le coût de financement du dollar, la demande de levier, et parfois une prime de rareté.

Donc `F(T)` **existe** et il est **observable directement** sur le marché des futures. C'est infiniment mieux qu'un modèle `S e^((r−q)T)` : pas besoin de taux, pas besoin de dividende, pas d'hypothèse. C'est exactement l'argument du document `data_design_deribit_ml.md` (§2.1).

Aujourd'hui, `v_surface` ne l'utilise pas : `k_logm = ln(K / coalesce(underlying_price, index_perpetuel))` . En clair, **on prend le spot**. Sur les maturités courtes l'erreur est de l'ordre du basis (quelques dixièmes de %), sur 1 an elle vaut le portage complet.

(b) Il n'y a pas de taux sans risque unique

Le « taux sans risque » en crypto est flou : taux de prêt USDT/USDC, taux de financement du perp, taux implicite des futures. Trois sources, trois chiffres.

La bonne réponse est de ne pas en avoir besoin. Pour ajuster SSVI, il suffit de `F(T)` ; `D(T)` n'intervient que si l'on veut repricer en dollars. Et pour pricer une option inverse, l'actualisation se fait dans la **monnaie de règlement** (BTC ou ETH), où le taux est nul par construction (l'actif est son propre numéraire).

(c) Les options sont inverses — et la parité n'est pas celle qu'on croit

C'est le point technique le plus important de cette partie, et celui qu'un entretien peut te demander.

Un call inverse paie, **en BTC**, `max(1 − K/S_T, 0)`. Un put inverse paie `max(K/S_T − 1, 0)`. La différence des deux payoffs est **exactement** `1 − K/S_T`, sans max. En actualisant dans la monnaie de règlement (taux nul dans cette unité) :

```
C_coin − P_coin = 1 − K · E[1/S_T]
```

C'est **linéaire en K**, avec :

- **une ordonnée à l'origine égale à 1** — un contrôle de cohérence gratuit : si ta régression de parité sur des options inverses ne donne pas 1, tes mids sont faux (ou tu as mélangé call et put d'échéances différentes) ;
- une **pente égale à** `−E[1/S_T]`, donc un forward « inverse » `K* = 1/E[1/S_T]`, qui n'est **pas** `E[S_T]`.

À comparer à la version linéaire (actions) : `C − P = D(F − K)`, pente `−D`, ordonnée `D·F`. Les deux régressions se ressemblent ; les objets qu'elles estiment sont différents. Écrire `k = ln(K/F)` avec le mauvais `F` est l'erreur de 2 % qui ne se voit pas.

Ce que ça implique concrètement : sur Deribit, l'IV publiée est calculée dans le modèle inverse. La bonne méthode n'est pas de supposer, c'est de **vérifier** : recalculer `mark_price` à partir de `mark_iv` avec notre propre formule inverse, et regarder si l'écart est inférieur à un tick. Si oui, la convention est identifiée ; sinon, on a appris quelque chose.

(d) Conséquence sur le moteur C++

Tout le moteur est écrit pour un sous-jacent **linéaire en dollars** :

Maillon	Formule actuelle	Ce qu'il faudrait pour l'inverse
Black-76	`C = D(F N(d₁) − K N(d₂))`	payer en coin : `C_coin = (1/F)(…)` avec `d₁,d₂` sur `F/K` — **ou** travailler en dollars avec `P_usd = P_coin · S`
Dupire	densité en coordonnée `S`	densité du prix inverse, ou modèle en `1/S`
PDE	`∂V/∂τ = ½σ²V_xx + …` en `S`	EDP sur `1/S_T` (SDE d'un actif sous la mesure coin), payoff `max(1−K/S_T,0)`
SSVI	`w(k,T)`	**inchangé** : c'est une surface d'IV, elle ne dépend pas du numéraire, seule la définition de l'IV change

Le tableau ci-dessus est la réponse courte à « est-ce que ton moteur marche pour la crypto ? » : **SSVI oui, Dupire et la PDE pas encore.**

5.5 Trois options de conception pour le forward crypto

Option	Comment	Avantages	Inconvénients
A — spot du perpétuel (aujourd'hui)	`F = index_price` du perp	disponible en continu, zéro calcul	ce n'est pas un forward ; biais croissant en T
B — basis du perp, extrapolé	`F(T) = S·e^(c·T)` avec `c = ln(perp/spot)/T_perp`	simple, capture le portage	suppose un basis constant en maturité — faux au-delà de quelques semaines
C — futures datés par échéance (recommandé)	`F(T_i)` lu directement ; interpolation en `ln F` entre échéances	c'est le marché qui parle ; pas d'hypothèse	demande de collecter les futures (13 échéances BTC/ETH)
D — parité put-call inverse	régression `C−P = 1 − K·m` sur les strikes liquides	sans modèle, parfaitement aligné sur les quotes	bruité là où un seul côté cote ; dépend de la qualité des mids

Recommandation : **C en principal, D en contrôle**. Si `F(T)` des futures et `K*` de la parité diffèrent de plus de ~10 bp, il y a un problème de données (mids périmés, mauvais appariement) et il faut le voir **avant** de calibrer.

5.6 Le plan de bascule, étape par étape

Étape 1 — collecter le forward. Ajouter au collecteur les canaux/endpoints qui portent les futures datés (`get_book_summary_by_currency` en REST toutes les 60 s porte `estimated_delivery_price` et `underlying_index`, ou `ticker.{BTC-<expiry>}.100ms` pour les 13 futures). Écrire une table `silver.future_tick` (ou `silver.forward_point`) avec `(observed_at, underlying, expiry, future_price, index_price)`.

Étape 2 — construire la courbe. Une vue `gold.v_forward` qui, pour chaque `(underlying, observed_at)`, donne `F(T_i)` et un taux de portage implicite `c(T) = ln(F(T)/S)/T`. Contrôle : `c(T)` doit être lisse en `T` ; un saut signale une échéance mal appariée.

Étape 3 — corriger `k`. Remplacer dans `v_surface` : `k = ln(K / F(T))` avec `F(T)` interpolé de `gold.v_forward`. **C'est le changement qui a le plus d'effet sur la qualité du fit**, et il ne touche pas une ligne de C++.

Étape 4 — le contrat d'entrée du moteur. Créer un export `gold.v_surface + gold.v_forward → data/gold/<snapshot_id>.csv` (ou JSON), avec, pour chaque ligne : `T, k, iv_pct, bid_iv, ask_iv, strike, option_type, expiry, snapshot_id`. Remplacer `MarketData::load` par un `loadSurfaceSnapshot` qui lit ce fichier. **Une seule classe à écrire, le reste du moteur ne change pas** — `calibrateSSVI` ne demande que `(k_grid, T_grid, iv_matrix)`.

Étape 5 — vérifier la convention d'IV. Sur un snapshot, reconstruire `mark_price` depuis `mark_iv` avec (i) Black-76 linéaire, (ii) Black inverse, et comparer au `mark_price` publié. Celle qui colle à moins d'un tick est la convention de la place. Écrire ce test.

Étape 6 — l'inverse dans le pricer. Deux voies possibles ; choisir explicitement :

- *voie dollar* : convertir les prix coin en dollars (`× S`), garder tout le moteur linéaire, et assumer que la surface d'IV utilisée est celle du modèle dollar — **ce n'est pas la même que celle de Deribit**, l'écart doit être chiffré ;
- *voie coin* : écrire le pricer inverse (payoff `max(1-K/S_T,0)`, numéraire coin), ce qui demande une EDP sur `1/S` et une Dupire dans la bonne coordonnée. Plus de travail, mais c'est la seule voie cohérente avec les quotes.

Recommandation : **voie coin** pour la cible, **voie dollar** uniquement comme étape de validation intermédiaire.

5.7 Résumé : ce qui est bon, ce qui est à changer

Sujet	Aujourd'hui	Cible crypto	Effort
Forward	spot (perp ou `underlying_price`)	`F(T)` des futures datés	1 migration + 1 vue + collecte
Taux `r`	plat 0 % ou courbe US	**inutile** pour le fit ; nul en unité coin	supprimer du chemin chaud
Dividendes	affine, moitié cash	sans objet (pas de dividende)	désactiver proprement
`D(T)`	`e^(-rT)`	1 en coin ; en dollar, déduit du basis	1 fonction
Convention IV	pourcentage, source Deribit	idem + test de cohérence prix/IV	1 test
Options inverses	ignorées	conversion explicite (§5.6 étape 6)	chantier moyen
Temps	365,25, 08:00 UTC	idem	rien à faire
PDE	linéaire en S	inverse (1/S) ou dollar assumé	chantier lourd

Ordre de priorité, si tu ne fais qu'une chose : **le forward (étapes 1-3)**. C'est le meilleur rapport effet/effort de tout le projet, et c'est ce qui rendra le premier fit BTC interprétable.

---

6. Cours SSVI, exercices et questions d'entretien

Cette partie est autonome : elle se relit seule avant un entretien. Les notations sont celles du code (§4.1).

6.1 Le problème que SSVI résout

Le marché cote une centaine de strikes par échéance et une vingtaine d'échéances. On veut une **fonction** `w(k,T)` définie partout, qui :

1. **repasse par les prix** (fit) ;
2. **ne viole aucun arbitrage** (contraintes) ;
3. **se comporte raisonnablement dans les ailes** (extrapolation) ;
4. **ait peu de paramètres** (stabilité dans le temps, interprétabilité).

Les quatre exigences sont en tension. Un spline 2D passe par les points (1) mais viole (2) et (4). Un modèle à 50 paramètres fit parfaitement et n'est plus une surface, c'est un bruit lissé.

La réponse de Gatheral : partir d'une forme fermée dont on sait démontrer l'absence d'arbitrage, et n'ajuster que quelques paramètres par-dessus.

6.2 De SVI à SSVI

SVI brut (une tranche, `k` mesuré depuis le forward) :

```
w(k) = a + b·( ρ(k − m) + √((k − m)² + σ²) )
```

Cinq paramètres par échéance : `a` (niveau), `b` (pente des ailes), `ρ` (asymétrie), `m` (décalage), `σ` (courbure ATM). Excellent fit, **mais** :

- 5 paramètres × 20 échéances = 100 paramètres à caler, avec des contraintes d'arbitrage **entre** échéances difficiles à exprimer ;
- rien ne garantit que les tranches soient cohérentes entre elles (arbitrage calendaire) ;
- le fit devient instable d'un snapshot à l'autre (le fameux « SVI butterfly problem »).

SSVI (Gatheral & Jacquier 2014) impose une structure :

```
w(k, θ) = θ/2 · [ 1 + ρ·φ(θ)·k + √( (φ(θ)·k + ρ)² + 1 − ρ² ) ]
```

avec **trois** objets seulement : `θ = θ(T)` (niveau ATM), `ρ` (asymétrie, constant), `φ(θ)` (pente de l'aile, fonction du niveau). Le projet utilise la paramétrisation « power law » :

```
φ(θ) = η / [ θ^γ (1 + θ)^(1−γ) ],      θ(T) = ν·T
```

soit **quatre nombres** pour toute la surface : `(ρ, η, γ, ν)`.

Pourquoi c'est puissant : la condition d'absence d'arbitrage papillon devient une condition sur `θ` et `φ` **seules**, valable pour tout `k` — donc pour toutes les échéances à la fois.

6.3 Interprétation des quatre paramètres

Paramètre	Rôle	Signe / valeur typique	Ce qui casse si on le contraint
ρ	asymétrie : pente à gauche vs droite	négatif (skew de put) ; SPX mesuré −0,54	impossible de reproduire un skew fort
η	échelle de φ, donc courbure globale	positif ; ~1,6 sur SPX	ailes trop plates ou trop raides
γ	vitesse d'aplatissement de φ en T	(0 ; 0,5] ; ~0,4-0,5 mesuré	structure par terme du skew figée
ν	variance ATM par unité de temps	$\nu = \sigma_{ATM}^2$; ~0,017 sur SPX (σ≈13 %)	niveau ATM forcé plat en T

Le point à retenir par cœur : $w(0,T) = \theta(T) = \nu \cdot T$, donc $\sigma_{ATM}(T) = \sqrt{\nu}$ **indépendant de T**. C'est le prix payé pour n'avoir que 4 paramètres.

6.4 Forme asymptotique : ce qui se passe dans les ailes

Pour $k \to \pm\infty$, $\sqrt{((\varphi k+\rho)^2 + 1-\rho^2)} \approx \varphi|k| \cdot (1 \pm \dots)$, donc

$$w(k) \approx (\theta\varphi/2)\cdot(1+\rho)\cdot k \quad (k \to +\infty)$$
$$w(k) \approx (\theta\varphi/2)\cdot(1-\rho)\cdot|k| \quad (k \to -\infty)$$

Le total variance croît **linéairement** en k. C'est la borne de Roger Lee ($w(k) \approx 2|k|$ au plus) : un SSVI admissible a $\theta\varphi(1\pm\rho)/2 \le 2$, ce qui est précisément contenu dans les conditions du théorème 4.2 (en plus strict).

Conséquence pratique : **l'extrapolation dans les ailes est une droite**. Si le marché a des ailes plus convexes (crypto, penny options), SSVI sous-estimera les extrêmes. C'est un argument pour des variantes (WSVI/SVI-jump-Wings) — à garder en réserve pour un entretien.

6.5 Arbitrage : les deux conditions

Arbitrage calendaire (calendar spread)

Définition : il y a arbitrage si la variance totale décroît avec la maturité à strike fixe. Intuition : $w(k,T)$ est un prix de variance ; elle doit croître avec le temps, sinon on vend la longue et on achète la courte pour un gain sans risque.

Condition (Gatheral-Jacquier, Thm 4.1) : pour tout θ,

$$\partial_\theta (\theta\varphi(\theta)) \ge 0 \qquad \text{et} \qquad 0 \le \partial_\theta(\theta\varphi(\theta)) \le (\varphi(\theta)/\rho^2)\cdot(1 + \sqrt{(1-\rho^2)})$$

(et $\theta(T)$ croissant).

Pour la famille $\varphi = \eta\, \theta^{(-\gamma)}(1+\theta)^{(\gamma-1)}$:

$$\theta\varphi = \eta\cdot\theta^{(1-\gamma)}/(1+\theta)^{(1-\gamma)} = \eta\cdot(\theta/(1+\theta))^{(1-\gamma)}$$

$$d(\theta\varphi)/d\theta = \eta(1-\gamma)\cdot(\theta/(1+\theta))^{(-\gamma)}\cdot 1/(1+\theta)^2$$

$$\Rightarrow \quad [d(\theta\varphi)/d\theta] \,/\, \varphi = (1-\gamma)/(1+\theta) \le 1-\gamma < 1$$

Or la borne de droite vaut $(1+\sqrt{(1-\rho^2)})/\rho^2 \geq 1$ pour tout $|\rho| < 1$. **La seconde condition est donc automatique** : pour cette famille, il suffit de $\nu > 0$ (pour que θ croisse) et $\gamma < 1$. Rien à programmer. C'est un argument de vente de SSVI en entretien.

Arbitrage papillon (butterfly)

Définition : il y a arbitrage si la densité risque-neutre implicite est négative quelque part, c'est-à-dire si le prix d'un papillon devient négatif.

La condition exacte est celle de **Durrleman** :

```
g(k) = (1 − k·w_k/(2w))² − (w_k²/4)·(1/w + 1/4) + w_kk/2 ≥ 0     pour tout k
```

Le lien avec Dupire — à savoir absolument : le dénominateur de la formule de Dupire est **exactement** `g(k)`. Donc

```
σ_loc²(k,T) = w_T / g(k)
```

et « densité négative » = « volatilité locale imaginaire » = « surface non arbitrable ». Ce n'est pas une coïncidence : les deux conditions disent la même chose, vue par le prix ou vue par la densité.

Les conditions suffisantes de Gatheral-Jacquier (Thm 4.2)

Pour la famille SSVI, il suffit que, pour tout θ :

```
θ·φ(θ)·(1 + |ρ|) < 4                    (condition 1)
θ·φ(θ)²·(1 + |ρ|) ≤ 4                   (condition 2)
```

Les deux suprema sont explicites :

```
sup_θ  θφ(θ)   = η                                (atteint quand θ → ∞)
sup_θ  θφ(θ)²  = η²·c(γ),   c(γ) = a^a/(1+a)^(1+a), a = 1−2γ
```

D'où **la borne utilisée dans le code** :

```
η_max(γ,ρ) = min( 4/(1+|ρ|),  2/√(c(γ)(1+|ρ|)) )
```

Trois nuances que 90 % des candidats ratent :

1. ces conditions sont **suffisantes, pas nécessaires** : les violer ne prouve pas qu'il y a arbitrage. Sur SPX, `η/η_max = 0,9999` (« non certifiable ») alors qu'un scan direct de `g` donnait `min g = 0,118` (aucun arbitrage) ;
2. $\gamma > \frac{1}{2}$ rend `sup θφ²` **infini** (car `a < 0`), donc la condition 2 est inatteignable : d'où la borne $\gamma \leq 0,5$ dans l'optimiseur, qui est une conséquence **du choix de paramétrisation**, pas de la théorie SSVI ;
3. le reparamétrage $\eta = u \cdot \eta_{max}(\gamma,\rho)$ transforme les deux inégalités en **une seule borne de boîte** $u \in [0,1]$, indispensable parce que BOBYQA refuse les contraintes non linéaires.

6.6 Calibration : ce qu'on optimise et pourquoi c'est délicat

```
L(ρ, η, γ, ν) = Σ_i Σ_j [ σ_SSVI(k_j, T_i) − σ_mkt(i,j) ]²
```

Quatre remarques :

1. **Pourquoi en vol et non en variance totale ?** En vol, l'erreur est comparable entre maturités (une erreur de 1 point de vol « vaut » pareil à 1 semaine et à 2 ans). En variance totale, les maturités longues dominent mécaniquement. En prix, les options chères dominent. Aucun n'est « juste » : c'est un choix de pondération, à assumer.
2. **Pourquoi pas pondérer par vega ?** Pondérer par vega revient à fitter en prix. Pour du pricing de book, c'est souvent mieux ; pour obtenir une **surface propre**, moins. Un bon compromis : poids `1/max(vega, ε)` ou poids par l'inverse du spread.
3. **Identification.** Sur une fenêtre de `k` étroite (par exemple `[−0,39 ; +0,065]` sur SPX), `ρ` et `η` sont **quasi colinéaires** : augmenter `η` et baisser `|ρ|` produit presque le même smile. D'où des paramètres qui sautent d'un snapshot à l'autre alors que la surface est stable. Remède : fitter sur une fenêtre large, ou régulariser.
4. **Coller à la borne.** `u → 1` signifie : « à fenêtre de données et famille donnée, le modèle voudrait plus de courbure que l'arbitrage ne l'autorise ». Diagnostic dans l'ordre : (a) le paramétrage est trop rigide (`θ = νT`), (b) l'objectif est mal pondéré, (c) le prétraitement (interpolation, grille commune) déforme, (d) **et seulement ensuite** le solveur.

6.7 La faiblesse structurelle à savoir défendre : `θ(T) = ν·T`

Ce choix force la variance ATM linéaire. Sur une vraie surface, la structure par terme ATM est souvent :

- **croissante puis plate** (actions calmes),
- **inversée** (stress, crypto en bull),
- **bombée** (autour d'événements).

Mesure faite sur SPX (20 échéances, 11 nœuds en `k`) : RMSE global 1,6 point de vol, mais l'erreur ATM passe de 0,8 pt à 3 mois à 2,7 pt à 10 mois, et `u = 0,9999`. Le test synthétique récupère les paramètres à 1e−6 → **ce n'est pas l'optimiseur, c'est le modèle**.

Le correctif standard (et ce qu'il faut savoir décrire) : libérer `θ` par échéance.

```
θ_i = w_mkt(0, T_i) = σ_ATM(T_i)²·T_i        (lu au marché, pas ajusté)
contrainte : θ_i non décroissant en i        (Thm 4.1a)
on n'ajuste plus que (ρ, η, γ)               (3 paramètres)
```

Avantages : la structure par terme ATM est exacte par construction ; le fit ne peut plus « tricher » en déformant `ν` ; `η/η_max` devient un vrai indicateur de tension. Coût : il faut un ATM propre (interpolation en `k` autour de zéro, ou un vrai strike ATM par échéance), et la contrainte de monotonie devient un problème d'optimisation sous contrainte (projection ou reparamétrage cumulatif : `θ_i = θ_1 + Σ Δ_j` avec `Δ_j ≥ 0`).

6.8 Dupire et volatilité locale : la dérivation en deux lignes

Deux résultats classiques :

1. **Breeden-Litzenberger** : la densité risque-neutre est la dérivée seconde du prix en strike : `q(K) = ∂²C/∂K² / D`.

2. **Dupire** : si tous les calls européens de tous strikes et maturités sont disponibles, il existe une diffusion locale unique qui les reproduit :

   $\sigma_{loc}^2(K,T) = (\partial C/\partial T) \,/\, (\tfrac{1}{2}K^2\,\partial^2 C/\partial K^2)$

En passant en variance totale `w = σ²T` et en log-moneyness, on obtient la forme utilisée dans le code :

```
σ_loc²(k,T) = w_T / g(k)      avec g la fonction de Durrleman (§6.5)
```

Ce qu'il faut savoir dire : la volatilité locale n'est pas une quantité observable, c'est une **représentation** : c'est la diffusion qui reproduit exactement les prix européens. Elle est notoirement instable (dérivée en T et seconde dérivée en K d'une surface estimée), et c'est précisément pour ça qu'on part d'une surface paramétrique lisse (SSVI) plutôt que d'un spline sur les points bruts.

6.9 Tableau de diagnostic : un symptôme → une cause

Symptôme observé	Cause la plus probable	Où regarder
`u = η/η_max` collé à 1	paramétrage trop rigide (`θ = νT`)	§6.7
résidu ATM qui croît avec `T`	structure par terme ATM forcée plate	§6.7
résidu concentré sur une aile	forward faux (décalage de `k`)	§5.5
résidu qui « saute » d'une échéance à l'autre	interpolation / grille commune / slice mal filtrée	§6.6 (4)
paramètres instables entre deux snapshots voisins	`ρ` / `η` quasi colinéaires + pas de warm start	§6.6 (3)
RMSE divisé par 10 après un changement de code	un `−1` ou un `NaN` avait été injecté avant	D2, et le bug de la grille en phase 1
σ_loc négatif ou NaN	arbitrage papillon non contraint	§6.5
prix PDE faux et **de pire en pire** quand on raffine	correction de Dirichlet oubliée dans le rhs	§3.5

Ce tableau vaut un vrai savoir-faire : en entretien, « comment déboguez-vous une calibration ? » se répond avec des symptômes, pas avec des principes.

6.10 Exercices (avec solutions)

E1. Montrer que `w(0,T) = θ(T)` et en déduire `σ_ATM(T)` .

> En `k = 0` : `√((0+ρ)² + 1−ρ²) = 1` , donc `w = θ/2·(1 + 0 + 1) = θ` . Puis `σ_ATM = √(w/T) = √(νT/T) = √ν` . **Indépendant de T** : c'est la limite du choix `θ = νT` .

E2. Calculer les pentes asymptotiques de `w` en `k → ±∞` et vérifier qu'elles sont finies.

> `√((φk+ρ)²+1−ρ²) ≈ φ|k|(1 + ρk/(φ|k|²)·…)` . À l'ordre dominant : `w ≈ θφ(1±ρ)|k|/2` . La pente en vol est `√(w/T) ~ √|k|` : les ailes d'IV croissent en racine de `|k|` — cohérent avec la borne de Lee.

E3. Démontrer que la seconde condition du Thm 4.1 est automatique pour `φ = ηθ^(−γ)(1+θ)^(γ−1)` .

> `θφ = η(θ/(1+θ))^(1−γ)` . Poser `x = θ/(1+θ)` , `dx/dθ = 1/(1+θ)²` . Alors `d(θφ)/dθ = η(1−γ)x^(−γ)/(1+θ)²` et `φ = η x^(−γ)/(1+θ)` . Donc `[d(θφ)/dθ]/φ = (1−γ)/(1+θ) ≤ 1−γ < 1` . Or `(1+√(1−ρ²))/ρ² ≥ 1` pour `|ρ|<1` . CQFD.

E4. Calculer `η_max` pour `γ = 0,4` , `ρ = −0,6` , puis pour `ρ = −0,9` . Commenter.

> `a = 0,2` ; `c = 0,2^(0,2)/1,2^(1,2) = e^(0,2 ln0,2)/e^(1,2 ln1,2) = 0,7248/1,2446 = 0,582`. `ρ=−0,6` : `min(4/1,6 ; 2/√(0,582·1,6)) = min(2,5 ; 2/0,965) = min(2,5 ; 2,072) = 2,072`. `ρ=−0,9` : `min(4/1,9 ; 2/√(0,582·1,9)) = min(2,105 ; 1,901) = 1,901`. Le skew fort (`|ρ|` grand) **réduit** le `η` admissible : c'est le couplage qui rend le fit tendu sur les actifs très skewés.

E5. On se trompe de forward : `F' = F(1+δ)`, `δ = 0,02`. Quel est l'effet sur `k` et sur `ρ` ?

> `k' = ln(K/F') = k − ln(1+δ) ≈ k − 0,02`. Le smile est translaté de `+0,02` vers la gauche. Comme le skew est `∂σ/∂k < 0` (skew de put), une translation vers la gauche **augmente** la pente apparente côté puts et la **diminue** côté calls : l'ajustement compense en augmentant `|ρ|`. Conclusion : un forward faux se manifeste par un `ρ` trop négatif — et c'est indiscernable d'un vrai skew, d'où l'importance de `F`.

E6. Calculer `g(θ)` pour SSVI en fonction de `(θ, ρ, φ)` et montrer que `g(θ) > 0` n'est pas automatique.

> `w(θ)=θ`, `w_k(θ) = θρφ`, `w_kk(θ) = θφ²(1−ρ²)/2`. Donc `g(θ) = 1 − θρ²φ²(1/4 + θ/16)·…` (le terme en `k` s'annule) : `g(θ) = 1 − (w_k²/4)(1/θ + 1/4) + w_kk/2 = 1 − θρ²φ²(1/4 + θ/16) + θφ²(1−ρ²)/4`. Le terme de gauche (en `ρ²φ²θ`) peut dépasser 1 : c'est l'arbitrage ATM, celui qui apparaît quand `η` est grand et `|ρ|` proche de 1 — exactement ce que borne `η_max`.

E7. Pourquoi le test synthétique doit-il être lancé **avant** de lire un fit réel ?

> Parce qu'un mauvais fit réel est ambigu : données, forward, grille, modèle, optimiseur. Un aller-retour synthétique isole le modèle + l'optimiseur : s'il récupère les paramètres à 1e−6, tout échec ultérieur vient de l'amont (données, forward) ou de la rigidité du modèle — plus de l'implémentation.

E8. On observe `u = 0,9999` et une RMSE de 1,6 pt. Quelles sont les deux hypothèses concurrentes, et quelle expérience les sépare ?

> H1 : le paramétrage est trop rigide (`θ=νT`). H2 : l'objectif/les données sont mal préparés (forward, grille, pondération). Expérience : libérer `θ` par échéance en le lisant au marché (3 paramètres restants). Si la RMSE chute et que `u` s'éloigne de 1 → H1. Si rien ne bouge → H2, et il faut regarder le forward et les résidus par aile.

6.11 Questions d'entretien (avec réponses courtes)

Q1. Pourquoi SSVI plutôt qu'un SVI par tranche ? Un SVI par tranche fit mieux mais ne garantit rien entre les tranches (arbitrage calendaire) et donne 5 paramètres par échéance, instables dans le temps. SSVI donne une garantie globale avec 4 paramètres interprétables.

Q2. Que contrôle chaque paramètre ? `ρ` l'asymétrie, `η` l'échelle de la pente des ailes, `γ` la vitesse d'aplatissement du skew avec la maturité, `ν` la variance ATM (et donc `σ_ATM = √ν`).

Q3. Comment garantissez-vous l'absence d'arbitrage ? Conditions suffisantes de Gatheral-Jacquier (Thm 4.1 calendaire, Thm 4.2 papillon), réduites à une borne de boîte sur `η = u·η_max(γ,ρ)` avec `u ≤ 1`. Le calendaire est automatique pour cette famille de `φ` (démontré, E3).

Q4. Vos conditions sont-elles nécessaires ? Non. Elles sont suffisantes : les violer ne prouve pas qu'il y a arbitrage. Sur notre fit SPX, `u ≈ 1` alors qu'un scan de Durrleman donne `min g = 0,118 > 0`. On rapporte donc `u` comme un **indicateur de tension**, pas comme un verdict.

Q5. Différence entre arbitrage calendaire et papillon ? Calendaire : la variance totale décroît avec `T` à strike fixe (prix de variance incohérent). Papillon : densité risque-neutre négative (prix de papillon négatif). Le premier se teste sur `θ(T)`, le second sur `g(k)` — qui est aussi le dénominateur de Dupire.

Q6. Pourquoi BOBYQA et pas un Levenberg-Marquardt ? Le résidu est une fonction lisse de `(ρ,η,γ,ν)` mais le gradient analytique de la chaîne complète n'est pas codé, et la dimension est 4. Un quasi-Newton à région de confiance sans gradient est adapté ; LM demanderait le jacobien (4 colonnes × N points), faisable mais inutile ici.

Q7. Pourquoi optimiser en log-moneyness et non en strike ? Parce que `k = ln(K/F)` rend le smile comparable entre maturités et entre sous-jacents, et parce que le modèle SSVI est écrit dans cette coordonnée. En strike, l'échelle change avec `F` et le problème est mal conditionné.

Q8. Comment obtenez-vous le forward en crypto ? Il est observable : futures datés (13 échéances) et perpétuel. On interpole en `ln F` entre échéances. En contrôle, la parité put-call **inverse** : `C − P = 1 − K·E[1/S_T]`, dont l'ordonnée à l'origine doit valoir exactement 1 — un test gratuit de qualité de données.

Q9. Quelle est la différence de parité entre options linéaires et inverses ? Linéaire : `C−P = D(F−K)` (pente `−D`, ordonnée `D·F`). Inverse : `C−P = 1 − K·E[1/S_T]` (pente `−E[1/S_T]`, ordonnée **1**). L'objet estimé n'est pas `F` mais `1/E[1/S_T]`.

Q10. Votre moteur marche-t-il pour des options inverses ? SSVI oui (c'est une surface d'IV). Non pour Dupire et la PDE : ils sont écrits pour un payoff linéaire en dollars. Il faut soit convertir en dollars et assumer une autre convention d'IV, soit écrire le pricer inverse (numéraire coin). C'est un chantier identifié, pas un oubli.

Q11. Comment inversez-vous la volatilité implicite ? Par Newton, mais protégé : la fonction prix est strictement croissante en σ, donc un encadrement suffit ; on n'accepte le pas de Newton que s'il reste dans l'intervalle, sinon on bissecte. On rejette d'abord les prix hors bornes d'arbitrage, avec une erreur **distincte** d'un échec de convergence.

Q12. Pourquoi certaines IV sont-elles inaccessibles ? Près des bornes, vega → 0 : l'information disparaît. Une option ITM profonde à court terme a une valeur temps sous la précision du double. Dans les ailes, une erreur de prix minuscule donne une erreur de vol énorme (σ mal conditionnée).

Q13. Énoncez Dupire. `σ_loc²(K,T) = (∂_T C)/(½K² ∂²_K C)` pour des calls européens. En variance totale : `σ_loc² = w_T / g(k)`, avec `g` la fonction de Durrleman. Intuition : la vol locale est le quotient du rythme d'accumulation de variance par la densité de strikes.

Q14. Pourquoi la volatilité locale est-elle instable ? Elle dépend d'une dérivée première en `T` et d'une dérivée seconde en `K` d'une surface *estimée*. Le bruit d'estimation est amplifié. D'où l'intérêt de partir d'une surface paramétrique lisse.

Q15. Expliquez votre schéma numérique. Crank-Nicolson (`θ=½`, ordre 2 en temps et en espace) sur une grille uniforme en `x = ln(S/K)`, large de ±5σ√T. Le système tridiagonal est résolu par Thomas en `O(N)`. Conditions de Dirichlet aux bords, avec correction du second membre. L'exercice américain est traité par projection (`max(V, intrinsèque)`) à chaque pas.

Q16. Quels sont les pièges d'un tel schéma ? Oublier la correction de Dirichlet (prix faux qui empire quand on raffine), le « ringing » de CN sur un payoff non lisse (parade : Rannacher), l'absence de détection de pivot nul, et la gestion des dates de détachement de dividende (décalage de grille).

Q17. Comment validez-vous la chaîne complète ? Trois tests, dans cet ordre : (1) aller-retour synthétique SSVI (paramètres récupérés à 1e–6) ; (2) PDE à vol plate = Black-Scholes analytique (erreur < 1e–2), et américain ≥ européen ; (3) parité put-call et bornes d'arbitrage sur les données réelles. Tant que (1) et (2) ne passent pas, aucun diagnostic sur données réelles n'est interprétable.

Q18. Votre RMSE est de 1,6 point de vol. Est-ce bon ? Ça dépend de l'usage. Pour de la représentation de surface et du pricing à la monnaie, c'est acceptable. Pour du pricing d'ailes, non : les 1,6 pt sont concentrés dans les ailes, où vega est faible — l'erreur en P&L est donc plus petite qu'elle n'en a l'air. Le chiffre honnête est : RMSE par échéance, et séparément ATM vs ailes.

Q19. Que feriez-vous la première semaine sur ce projet ? Brancher la base sur le moteur (export gold → fichier d'entrée), corriger le forward (futures datés), lancer un fit BTC avec diagnostic par échéance. Puis libérer `θ` par échéance si `u` colle à 1. Rien d'autre avant d'avoir ces trois mesures.

Q20. Quel est le plus gros risque du projet aujourd'hui ? Pas le modèle : la **coupure** entre l'usine à données et le moteur. Tant que le moteur lit un fichier figé, chaque conclusion quantitative est non testable en continu, et les bugs silencieux (unités, forward, sentinelles) reviennent.

---

7. Feuille de route et critères d'acceptation

Quatre lots, dans cet ordre. Chaque lot se termine par un **critère mesurable**, pas par une impression.

Lot A — Raccorder (le plus rentable)

#	Tâche	Critère d'acceptation
A1	Ajouter la collecte des futures datés + `estimated_delivery_price`	une ligne par échéance et par 5 min dans une table, `c(T)` lisse en T
A2	Vue `gold.v_forward` + correction de `k_logm`	`k` recalculé avec `F(T)`, écart médian vs ancien `k` documenté
A3	Export `gold.v_surface + v_forward` → fichier d'entrée moteur	un fichier par snapshot, avec `snapshot_id` et hash
A4	`loadSurfaceSnapshot` côté C++ (remplace `MarketData::load`)	`calibrate_spx_minimal` calibre un snapshot BTC sans aucune retouche
A5	Nouvelle migration `calibration_run` / `calibration_slice` / `quote_fit` au format crypto	un run écrit, relu, avec paramètres et résidus
A6	Premier fit réel BTC + ETH, diagnostic par échéance	un tableau : T, n, RMSE, RMSE ATM, RMSE ailes, `u`

Critère du lot : on peut dire « voici la RMSE du modèle sur BTC au 28 octobre, par échéance » et le refaire à l'identique depuis le `snapshot_id`.

Lot B — Rendre le moteur croyable

#	Tâche	Critère
B1	Newton protégé + bornes d'arbitrage dans `ImpliedVol`	test sur 60 combinaisons (K,T,σ), échecs seulement en aile extrême
B2	**Vrais** tests C++ : `test_implied_vol`, `test_market_data`, `test_pde_pricer`	`ctest` lance 5 tests qui échouent si on casse le code
B3	Supprimer ou réparer `test_ssvi_synthetic` (aujourd'hui vert à vide)	le test affiche des paramètres récupérés, pas « TODO »
B4	`σ_loc` : remplacer l'écrasement NaN par une erreur	une surface arbitrageuse fait échouer le run, bruyamment
B5	`θ` par échéance (lu au marché, contrainte de monotonie)	`u` s'éloigne de 1, RMSE ATM ne croît plus avec T
B6	Validation PDE : vol plate vs BS, américain ≥ européen	erreur < 1e−2, test automatisé

Critère du lot : les trois tests de la §6.11 Q17 passent, et le tableau de diagnostic ne montre plus de motif systématique.

Lot C — Crypto-complet

#	Tâche	Critère
C1	Identifier la convention d'IV de Deribit (recalculer `mark_price`)	écart < 1 tick
C2	Pricer inverse (numéraire coin) ou conversion dollar explicite	parité inverse vérifiée sur les prix recalculés
C3	Dupire dans la coordonnée choisie	σ_loc > 0 partout sur la surface calibrée
C4	Calibration planifiée (15 min) + warm start	deux runs consécutifs : paramètres continus, temps de fit divisé
C5	Monitoring des fits (RMSE, `u`, dérive des paramètres)	alerte si RMSE > seuil

Lot D — Industrialisation

Moteur dans l'image Docker (multi-stage arm64), calibration dans le pipeline, rétention du bronze, vérification mensuelle de restauration, table de comparaison entre runs.

---

8. Annexes

8.1 Index des fichiers (une ligne chacun)

Fichier	Une ligne
`include/pricing/blackscholes.hpp` / `src/pricing/blackscholes.cpp`	BS spot et Black-76 forward, vega
`include/calibration/ImpliedVol.hpp` / `src/...`	prix → σ, avec `variant` valeur/erreur
`include/calibration/SSVI.hpp` / `src/...`	la surface, ses dérivées, les conditions d'arbitrage, la calibration
`include/calibration/LocalVol.hpp` / `src/...`	Dupire en variance totale
`include/calibration/FitDiagnostics.hpp` / `src/...`	RMSE global/par échéance, `η/η_max`
`include/pde/Grid.hpp` / `src/pde/Grid.cpp`	grille uniforme en `ln(S/K)`
`include/pde/Tridiag.hpp` / `src/pde/Tridiag.cpp`	assemblage Crank-Nicolson + Thomas
`include/pde/PDEPricer.hpp` / `src/pde/PDEPricer.cpp`	boucle en temps, bords, exercice américain, greeks
`include/data/RateCurve.hpp` / `src/...`	courbe de taux zéro, interpolation linéaire
`include/data/DivCurve.hpp` / `src/...`	dividendes affines `(α,β)`, PV, forward discret
`include/data/MarketData.hpp` / `src/...`	chargement CSV hérité (spot, taux, divs, quotes)
`include/data/DateUtils.hpp` / `src/...`	dates → années (365,25)
`include/opt/NelderMead.hpp` / `src/...`	Nelder-Mead incomplet, non utilisé
`apps/calibrate_spx_minimal.cpp`	prototype de bout en bout, sortie JSON
`apps/calibrate_spx_main.cpp`	runner « propre », encore en prose
`src/main.cpp`	bac à sable historique (tests noyés + pipeline SPX)
`python/collect/deribit_ws.py`	collecteur WebSocket → bronze
`python/collect/names.py`	parsing des noms d'instruments
`python/collect/analyze_bronze.py`	rapport qualité du bronze (latence, trous, compression)
`python/jobs/load_marks.py`	bronze → silver, échantillonnage 5 min, idempotent
`python/db/migrate.py`	runner de migrations avec registre sha256
`python/db/connection.py`	connexion psycopg par variables d'environnement
`python/io/atomic.py`	écriture atomique de fichiers
`sql/001..008`	schéma, extensions, Deribit, retrait CBOE, unités, vues, prix de référence, couches
`scripts/db-tunnel.sh`	tunnel SSH vers la base du mini
`scripts/pull_dev.py`	extraction base → parquet + manifeste
`scripts/plot_surface.py`	smile et structure par terme depuis les vues
`scripts/macmini_bootstrap.sh`	installation du mini
`ops/`	cron : garde-fou stack, alertes, chargement, backups, restauration
`docs/master.md`	synthèse : architecture + chaîne mathématique

Fichier	Une ligne
`docs/cours_complet.pdf`	**ce document**

8.2 Commandes utiles

```
# moteur
cmake -S . -B build && cmake --build build -j4
ctest --test-dir build --output-on-failure
./build/calibrate_spx_minimal --json-out /tmp/fit.json

# python
./.venv/bin/python -m pytest tests/python -q
./.venv/bin/python -m python.db.migrate --status

# base (depuis le MacBook)
scripts/db-tunnel.sh start
psql -h 127.0.0.1 -p 15432 -U vse -d volsurface
./.venv/bin/python scripts/pull_dev.py --jours 2

# mini
ssh macmini-ai@macmini-ai.local
cd ~/vol-surface-engine && docker compose ps
crontab -l | grep vol-surface
```

8.3 Les TODO réels du dépôt (état de la revue)

Fichier	Contenu du TODO	Statut
`apps/calibrate_spx_main.cpp`	11 étapes en prose (groupe, filtre, grille, calibration, diagnostic)	à implémenter ou à supprimer au profit de `calibrate_spx_minimal`
`tests/test_ssvi_synthetic.cpp`	aller-retour synthétique	**stub qui passe** (danger)
`tests/test_implied_vol.cpp`, `test_market_data.cpp`, `test_pde_pricer.cpp`	prose sans `main`	non compilés
`src/pde/PDEPricer.cpp`	dividendes affines (3 marqueurs)	non fait
`src/opt/NelderMead.cpp`	critère d'arrêt `WIP`	abandonné de fait
`python/data/*` (supprimé)	ancienne chaîne CBOE	remplacé par Deribit

8.4 Ce que ce document n'est pas

- Ce n'est pas une spécification figée : plusieurs affirmations deviendront fausses au lot A (le forward, le lien base → moteur).
- Ce n'est pas un cours de finance de marché complet : il couvre ce que **ce projet** utilise. Les manques assumés : stochastique continue (le SDE sous-jacent n'est jamais posé explicitement), calibration américaine, et microstructure (spread, impact).

Fin du document.